

ALGORITHMS A.Y. 2025/2026

# Demand Response Transit

The province of Cosenza  
A CASE STUDY



## Demand Response Transit

A case study: the province of Cosenza

**Author:** Michele Vincenzo Branca

**Institute:** University of Turin and Collegio Carlo Alberto

**Date:** July 2, 2026



Research  
Education  
Outreach

**CCA**

*«All knowledge degenerates into probability»*

*- David Hume*

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Chapter 1: Introduction</b>                                | <b>1</b>  |
| 1.1 Problem presentation . . . . .                            | 1         |
| 1.2 Mathematical model . . . . .                              | 3         |
| 1.3 Datasets . . . . .                                        | 6         |
| <b>Chapter 2: Algorithm</b>                                   | <b>9</b>  |
| 2.1 Description . . . . .                                     | 9         |
| 2.2 Pseudocode and comments . . . . .                         | 9         |
| 2.3 A toy example . . . . .                                   | 12        |
| 2.4 Code . . . . .                                            | 36        |
| 2.5 Solving the toy example with the proposed code . . . . .  | 41        |
| 2.6 Solving the full dataset with the proposed code . . . . . | 42        |
| <b>Chapter 3: Analysis of the solution</b>                    | <b>44</b> |
| 3.1 Datastructures used . . . . .                             | 44        |
| 3.2 Asymptotic complexity . . . . .                           | 45        |
| <b>Chapter 4: Simulations</b>                                 | <b>48</b> |
| 4.1 Online setting . . . . .                                  | 48        |
| 4.2 Running time versus number of requests . . . . .          | 50        |
| 4.3 Fleet size versus acceptance . . . . .                    | 52        |
| <b>Chapter 5: Conclusion</b>                                  | <b>53</b> |
| <b>Bibliography</b>                                           | <b>56</b> |
| <b>Appendix: Constructing the datasets</b>                    | <b>57</b> |
| 1.1 cities_coordinates.txt . . . . .                          | 58        |
| 1.2 distance-duration_table.txt . . . . .                     | 58        |
| 1.3 requests.txt . . . . .                                    | 59        |

# Chapter 1: Introduction

## 1.1 Problem presentation

Demand Response Transit (hereafter, DRT) is a relatively new paradigm of transportation based on the idea that the supply within a transport system should be somewhat proportional to the demand of the passengers. Since it is not yet a well-established topic in scientific research, there is not a commonly accepted definition of it and of the terms to it related, which are often used in a deceiving and inappropriate manner. In presenting this topic, we will follow the analysis made in [2] and [7]. There are many motivations for such a new way of thinking about public transport. We can summarize them here as: greater convenience of using public transportation, emissions reduction, saving resources, guarantee of an efficient public transportation system even in rural areas without running into prohibitive costs.

There are a lot of challenges concerning DRT, both theoretically and on a more practical basis. Indeed, efficiency and effectiveness must be achieved, and a certain ease of usage ensured. This project is concerned with a province of Calabria, Cosenza, in which public transportation suffers from severe disfunctionality and is perceived as a slow and unreliable means compared to individual travel by car. We will consider the DRT paradigm applied to buses, given that Calabria is a fairly mountainous region and the railway transportation system is not so much developed (exception made for the Tyrrhenian coast) because of this, buses offer a concrete alternative to individual transport. The problem of people's perception and more aspects commonly found when discussing public transportation in rural and low population density areas are more carefully analyzed in [7].

It is important to say that different choices can be made when discussing DRT in an urban framework and a rural one such the one we will be concerned with, and each of them leads to different situations. One might think of many problems related to DRT, namely:

- **bus scheduling**, with or without real-time customer insertion;
- **fleet sizing**, with known or unknown demand levels;
- **pricing**;
- **routing problems**, i.e. **Dial-a-Ride-Problem (DaRP)**;

etc.

Essentially, the routing part of a DRT system is the already mentioned DaRP, which generalizes the Vehicle Routing Problem adding precedence constraints between pick-ups and deliveries and person-specific requirements such as time windows and maximum ride times. The VRP is an NP-hard problem, thus the DaRP is itself NP-hard. As it is reported in [5], exact approaches remain confined to small instances, thus the generally much learger effort into heuristics. Two families can be distinguished:



**Figure 1.1:** Municipalities of the province of Cosenza.

- **metaheuristics** for the static problem, in which all requests are known in advance (see for example the tabu search of [4], designing the least costing routes serving all the requests – provided met capacity, route-duration and ride-time constraints –, and obtains high-quality solutions. This, though at the price of computation times incompatible with answering a customer in real time and thus for the creation of a web based service;
- **constructive** insertion heuristics, in which requests arrive over time and must be irrevocably accepted or rejected on the spot. The double dynamic algorithm of [3], which we adopt and describe in the following chapter, belongs to this family: this consists of a fast insertion step which provides an immediate answer to the customer and an optimization step that improves the found solution in the idle time between two arrivals.

Another field of research is interested in the structure of the network itself. In [6] the set of stations is first optimized by clustering the reservation demand (combining DBSCAN and K-means algorithms) and then solving the resulting scheduling model with a genetic simulated-annealing algorithm, an approach thought for low-demand connections between urban and rural areas. The solution implemented throughout this project borrows from the constructive method proposed in [3] and reformulates the idea proposed in [6] in a slightly different form: a demand-based clustering step (although a much simpler one) to place the depots is used, and an insertion-based heuristic to build the routes gives the overall solution.

Whichever the framework being considered, following [3], the answer to the question "does the system accept (possibly after a feasibility assessment) requests when the service is running?" makes it possible to distinguish two paths that the problem can take: if the answer is yes, we are in a "running mode" scenario, otherwise in a "booking mode" one. In practice, it is reasonable to think of a mixed model, in which part of the passengers have booked in advance and another part (latecomers!) book while the ride has already taken off.

Furthermore, the question of how routes should be traced should be addressed: should we create the tours, say, on a daily basis based on the requests that the passengers have made? Should we instead identify some fixed routes (perhaps analyzing requests made via polls) and then allow for deviations on the path [6]? In tracing the routes, what are our objectives? Minimizing costs? Travel times?

We will now answer to all of this. In applying this paradigm to the Calabrian case:

- we will consider a queue data-structure to deal with the requests, allowing to tackle both the "booking mode" (the one we will focus on) and the "running mode" (one might think, for example, of a web-based application);
- in tracing the routes, we will not individuate a set of fixed routes passing from some selected cities, rather following [3] we will select some cities as **depots**, i.e. cities from which the tours can begin. The tours will then be constructed based on the requests of the moment, exactly as the authors of the quoted article propose. We will not dwelve too deeply in how to select the depots: we will run a simple DBSCAN algorithm, which clusters towns weighted in some way (in our case, by demand, i.e. each city is weighted by how many times it appears as origin or destination in the `requests.txt` file of which we will talk later on), using a Python's library, given that this is somewhat part of data pre-processing; and then take the highest-demand city in a cluster as the representative one of the cluster itself;
- the objective will be minimizing customer delays;
- we will not be concerned with the pricing or the fleet sizing issue.

## 1.2 Mathematical model

Let us first present the general model in its three-index formulation as done in [5] and then specialize it to our case. The DaRP is formulated on a graph  $G = (V, A)$ . We begin considering the case in which we have a single depot:

### 1. Sets and variables.

- the set of all nodes  $V$  is partitioned in two copies of the depot  $\{0, 2n + 1\}$ , the set  $P = \{1, \dots, n\}$  of pick-up nodes and the set  $D = \{n + 1, \dots, 2n\}$  of delivery nodes;
- the set  $R = \{1, \dots, n\}$  of the transport requests, where  $i \in R$  is a couple  $(i, n + i)$ , where  $i \in P$ ,  $n + i \in D$ ;
- the arc set  $A = \{(i, j) : i = 0, j \in P \text{ or } i, j \in P \cup D, \text{ with } i \neq j \text{ and } i \neq n + j \text{ or } i \in D, j = 2n + 1\}$ ;
- the load  $q_i$  and the service duration  $d_i$  associated to each vertex  $v_i \in V$ , where  $q_i \geq 0$ ,  $q_i = -q_{i-n} \forall i = n + 1, \dots, 2n$  and  $d_i \geq 0$ ,  $d_0 = d_{2n+1} = 0$ ;
- the set of vehicles (the fleet)  $K$ . Each vehicle  $k \in K$  is based at the depot and performs a single route;
- the capacity of each vehicle  $k \in K$ ,  $Q_k$ ;
- the maximal duration of route  $k$ ,  $T_k$ ;
- the cost of traversing arc  $(i, j)$  with vehicle  $k$ ,  $c_{ij}^k$ ;
- the travel time (in minutes) between nodes  $i$  and  $j$  in  $V$ ,  $t_{ij} \geq 0$ . This quantity satisfies the triangle inequality;
- the maximal ride time  $L$ ;
- the time window of the service at vertex  $i$ ,  $[e_i, \ell_i]$ ;
- the indicator of the event "vehicle  $k$  traverses arc  $(i, j)$ ",  $x_{ij}^k \in \{0, 1\}$ ;
- the time at which vehicle  $k$  starts servicing vertex  $i$ ,  $u_i^k$ ;
- the load of vehicle  $k$  upon leaving vertex  $i$ ,  $w_i^k$ ;
- the ride time of user  $i$ ,  $r_i^k$ .

### 2. Objective. organize the trips so as to

$$\text{minimize } \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} c_{ij}^k x_{ij}^k$$

### 3. Constraints.

$$\forall i \in P, \quad \sum_{k \in K} \sum_{j \in V} x_{ij}^k = 1; \quad (1.1)$$

$$\forall k \in K, \quad \sum_{i \in V} x_{0i}^k = \sum_{i \in V} x_{i,2n+1}^k = 1; \quad (1.2)$$

$$\forall i \in P, k \in K, \quad \sum_{j \in V} x_{ij}^k - \sum_{j \in V} x_{n+i,j}^k = 0; \quad (1.3)$$

$$\forall i \in P \cup D, k \in K, \quad \sum_{j \in V} x_{ji}^k - \sum_{j \in V} x_{ij}^k = 0; \quad (1.4)$$

$$\forall i, j \in V, k \in K, \quad u_j^k \geq (u_i^k + d_i + t_{ij})x_{ij}^k; \quad (1.5)$$

$$\forall i, j \in V, k \in K, \quad w_j^k \geq (w_i^k + q_j)x_{ij}^k; \quad (1.6)$$

$$\forall i \in P, k \in K, \quad r_i^k \geq u_{n+i}^k - (u_i^k + d_i); \quad (1.7)$$

$$\forall k \in K, \quad u_{2n+1}^k - u_0^k \leq T_k; \quad (1.8)$$

$$\forall i \in V, k \in K, \quad e_i \leq u_i^k \leq \ell_i; \quad (1.9)$$

$$\forall i \in P, k \in K, \quad t_{i,n+i} \leq r_i^k \leq L; \quad (1.10)$$

$$\forall i \in V, k \in K, \quad \max\{0, q_i\} \leq w_i^k \leq \min\{Q_k, Q_k + q_i\}; \quad (1.11)$$

$$\forall i, j \in V, k \in K, \quad x_{ij}^k = 0 \text{ or } 1. \quad (1.12)$$

Constraints (1.1) and (1.3) ensure that each request is served once by the same vehicle, constraints (1.2) and (1.4) guarantee that each vehicle starts and ends its route at the depot, constraints (1.5) to (1.7) define starts of service times, vehicle loads and user ride times, respectively, while constraints (1.8) to (1.12) ensure that these will be feasible. Usually in equation 1.9 the time window is somewhat dependent on the customer's request.

We now specialize the model to our case:

#### 1. Additional set and variables.

- the set of **depots**  $\mathcal{D} \subseteq V$ ;
- the requested departure (pick-up) time of request  $i \in R$ ,  $\tau_i$ , so that  $i \in R$  is no more a couple  $(i, n+i)$ , where  $i \in P$ ,  $n+i \in D$ , but a triple  $(i, n+i, \tau_i)$ , where  $i \in P$ ,  $n+i \in D$ ;
- the pick-up tolerance  $W$ , i.e. the time a customer is willing to delay the departure of their trip;
- the indicator of acceptance of the request of customer  $i$ ,  $y_i \in \{0, 1\}$ ;
- the assignment of the vehicles to the depots,  $\rho : K \rightarrow \mathcal{D}$ , where  $\rho(k)$  is the depot at which vehicle  $k$  is based;
- the two depot copies  $o_k, o'_k$  for each vehicle  $k \in K$ , i.e. the start and end node of its route — both located at  $\rho(k)$ ;
- the ride-time tolerance  $\vartheta$ , i.e. the maximum detour a customer accepts beyond the direct travel time  $t_{i,n+i}$ ;
- the shift-duration is  $T$  for all vehicles (i.e., a common maximal durations for all routes);
- the service time  $d$  common to all nodes, except for the depots, for which we have  $d_{o_k} = d_{o'_k} = 0$ ;
- the capacity common to all vehicles,  $Q$ ;
- the beginning of each vehicle's shift,  $T_0$ .

2. **Objective.** organize the trips so as to

first maximize  $\sum_{i \in P} y_i$ , and

then minimize  $z = \sum_{i \in P: y_i=1} \left[ \underbrace{(u_i^k - \tau_i)}_{\text{pick-up delay}} + \underbrace{(u_{n+i}^k - \tau_i - t_{i,n+i})}_{\text{arrival delay}} \right] \forall k \in K.$

3. **(Modified) constraints.**

$$\forall i \in P, \quad \sum_{k \in K} \sum_{j \in V} x_{ij}^k = y_i; \quad (1.13)$$

$$\forall k \in K, \quad \sum_{i \in V} x_{o_k i}^k = \sum_{i \in V} x_{i, o'_k}^k = 1; \quad (1.14)$$

$$\forall i \in P, k \in K, \quad \sum_{j \in V} x_{ij}^k - \sum_{j \in V} x_{n+i, j}^k = 0; \quad (1.15)$$

$$\forall i \in P \cup D, k \in K, \quad \sum_{j \in V} x_{ji}^k - \sum_{j \in V} x_{ij}^k = 0; \quad (1.16)$$

$$\forall k \in K, \quad \begin{cases} u_j^k \geq (u_i^k + d + t_{ij})x_{ij}^k, & \forall i \in P \cup D, j \in V, \\ u_j^k \geq (u_i^k + t_{ij})x_{ij}^k, & \forall i \in \{o_k, o'_k\}, j \in V; \end{cases} \quad (1.17)$$

$$\forall i, j \in V, k \in K, \quad w_j^k \geq (w_i^k + q_j)x_{ij}^k; \quad (1.18)$$

$$\forall i \in P, k \in K, \quad r_i^k \geq u_{n+i}^k - (u_i^k + d); \quad (1.19)$$

$$\forall k \in K, \quad T_0 \leq u_{o_k}^k \text{ and } u_{o'_k}^k \leq T_0 + T; \quad (1.20)$$

$$\forall i \in P, k \in K, \quad \tau_i \leq u_i^k \leq \tau_i + W; \quad (1.21)$$

$$\forall i \in P, k \in K, \quad t_{i, n+i} \leq r_i^k \leq t_{i, n+i} + \vartheta; \quad (1.22)$$

$$\forall i \in V, k \in K, \quad \max\{0, q_i\} \leq w_i^k \leq \min\{Q, Q + q_i\}; \quad (1.23)$$

$$\forall i, j \in V, k \in K, \quad x_{ij}^k = 0 \text{ or } 1; \quad (1.24)$$

$$\forall i \in P, \quad y_i \in \{0, 1\}. \quad (1.25)$$

where the simplifications or modifications occur because we suppose, as it could have been inferred by the new set and variables, that:

- it might happen that a certain request is not accepted;
- vehicle  $k$  starts and end the trip at  $\rho(k)$ ;
- the service duration is equal for all nodes, i.e.  $d_i = d$  for some  $d$  for each  $i$ ;
- all trips have to happen in certain time slot (precisely  $[T_0, T_0 + T]$ );
- the time windows are now induced by the data and rather than on the node, they depend on the request:

$$[e_i, \ell_i] = [\tau_i, \tau_i + W],$$

where  $W$  is the pick-up tolerance;

- having specified the desired departure times, customers are all willing to depart  $W$  (minutes) later than said time;
- each customer accepts to travel  $\vartheta$  minutes more;
- the fleet has homogeneous capacities, meaning that  $Q_k = Q$  for some  $Q$ .

We can see that the equations are similar in spirit to those of the "basic" model, the differences are that we

have multiple depots to deal with; we have made some simplifying, yet realistically admissible, hypothesis and the objective is changed from the minimization of the cost to the maximization of the accepted requests and the minimization of the customers' delay.

In the Cosenza province case, the set of the nodes will consist of all the municipalities present in the province itself. Since we are concerned with intercity (rural) transport rather than urban transport, we will make the simplification that there is just one stop in each city, thus effectively identifying each city with a single point. The best thing to do would have been to gather the most important sites for each city in which a bus should arrive, but the time (and mental) budget was limited.

## 1.3 Datasets

One of the biggest challenges during the realization of this project has been the collection of the (minimal amount of) data which were needed to run the code implemented to (approximately) solve the presented problem. Most of the literature concerning DRT focuses in urban zones. Usually data for (big and, most importantly – unfortunately and unfairly –, rich) cities are easier to collect, or simply available online. This is obviously not the case for one of Italy's (poorest) provinces. Yet, it has been possible to construct some of the needed datasets from the sites of some national Italian institute (e.g., ISTAT) and to synthetically generate another part of them. We now describe this process thoroughly.

The first thing which needed to be done was to **gather the names of all cities in the province of Cosenza**. We can do this by exploiting the website [istat.it](http://istat.it) of the national statistical institute of Italy, which stores various data regarding the country. In particular, the following [link](#) can be used to download a `.csv` file containing all municipalities of Italy, with information on the province to which they belong to. After getting the results, one can observe that there are 150 municipalities in the province of Cosenza.

The next thing was to `geocode` them, i.e. **for each of the retrieved cities, individuate a point on Earth's surface which identifies said city**. It has been possible to do so thanks to the Python's library `geopy` OpenStreetMap data, whose geocoder `Nominatim` borrows from OpenStreetMap data – more details are provided in the Colab notebook. In this way, one is able to get a couple of coordinates – latitude and longitude – identifying a point which in turns identifies one of the previously retrieved municipalities. One might ask what is this point. Turns out that, for each city, this is the centroid (the point whose coordinates are the averages of the coordinates of the points which fall inside the polygon representing said administrative region) of the administrative region of the city itself – again, more details are given in the notebook. As we were mentioning before, this is (obviously) a simplifying assumption and one might opt to do something else, for example identify a couple of points of interest for each city and `geocode` them instead (the tools used also allow to retrieve the coordinates of more specific locations). The results have been saved in the file `cities_coordinates.txt`. Here we show a map of the municipalities of Cosenza marked with red circles of varying dimensions: the bigger, the higher the demand.

The next thing was to **get the road distances between each pair of cities**, as well as the duration of the trip. To do so we have used the OSRM (Open Source Routing Machine) API, a service provided using – again – OpenStreetMap data. Here we made the assumption that distances and durations are symmetric, meaning that if the road distance between city A and city B is  $x$  kilometers and the associated travel duration is  $y$  minutes, then  $x$  is the road distance between city B and city A and  $y$  is the associated travel duration. Even with this assumptions, one would still need to submit a lot of API requests ( $149 \times 148/2 = 11026$ ) which, if we consider the need to give the server a bit of time between two consecutive requests to avoid the server collapse,



immediately translates to a lot of potentially wasted time. Thankfully, OpenStreetMap developers thought about it and provided a service (namely, the `table` service) that allows for more complex API requests, able to yield the pairwise road distance and the associated travel duration within two groups of cities, the first one to be considered as sources and the second one to be considered as destinations. For more details, see the `.ipynb` file. The (upper triangular) matrix containing both road distances and the associated travel durations has been saved in the file `distance-duration_table.txt`. Because of the `table` service, this procedure does not take that much time.

Now comes the hard part of getting the data. Indeed, the model we described – and the code we will implement – strongly depends on **customers' demands**. Obviously, there is no dataset regarding the province of Cosenza which stores citizens demands concerning public transportation, thus a little bit of fantasy had to be used. The way in which we solved this problem was to look for a mathematical model able to provide reasonable fake requests made by fake customers, at least to run the algorithm we will implement to approximate a solution to the model proposed in the previous section. This is clearly not optimal since, as a well-known statistician – namely, George Edward Pelham Box – once said,

*«All models are wrong, [...]»*

yet it is the only sensible thing to do here, thus relying on the continuation of the quote:

*«[...] but some are useful.»*

we have decided to follow this idea and rely on some – very old – model. The latter has been proposed in 1946 by the American linguist George Kingsley Zipf, even though the conceptual beginning of it can be traced back to 1858 (see [1], section 4.2.1), and it is based on an equation resembling Newton's law of gravitation. More precisely, denoting with  $T_{ij}$  the migration flow between community  $i$  and community  $j$ :

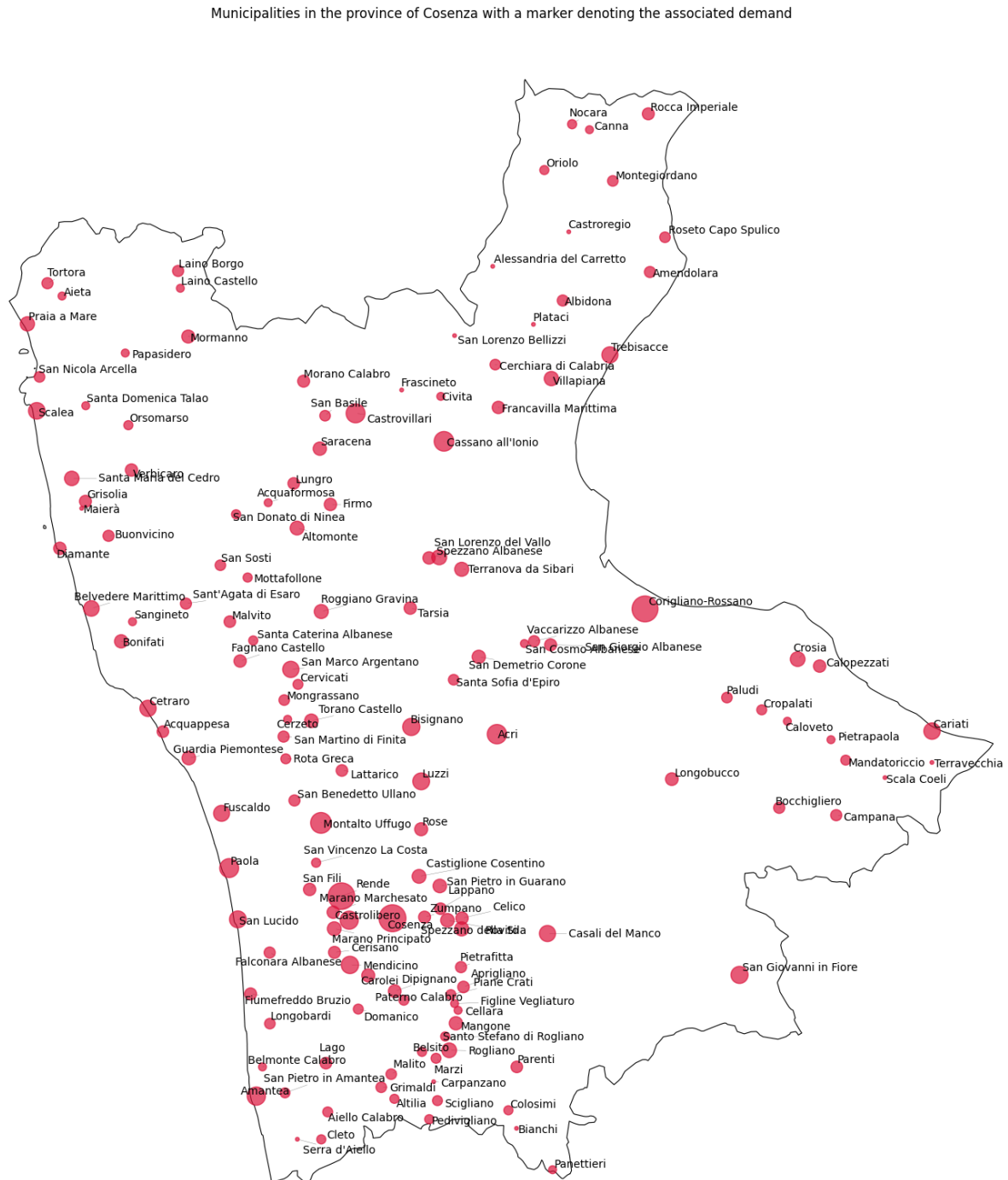
$$T_{ij} \propto \frac{P_i \cdot P_j}{r_{ij}},$$

where  $P_i$  is the population of community  $i$ ,  $P_j$  the one of community  $j$  and  $r_{ij}$  is the distance between community  $i$  and  $j$ . Generalizing:

$$T_{ij} = m_i \cdot m_j \cdot f(r_{ij}),$$

for a certain function  $f$  (which we will take to be of exponential form) and "masses" (depending on the populations)  $m_i$ ,  $m_j$ . These masses are related to the number of trips leaving  $i$  being attracted by  $j$ . In modelling them, one usually takes into account some socio-economic factors. We will take into account the population, the presence of universities, the presence of high-schools and the presence of railway stations (and the corresponding importance) in a certain way. More details are in the notebook file. So few factors have been taken into account because of time – and again, mental sanity – constraints, given that most of the factors had to be coded by hand. Having calculated the magnitude of our migratory flows between the communities represented by the municipalities in the province of Cosenza previously retrieved, we have simply constructed a probability distribution  $(p_{ij})_{i=1}^{150}$  depending on them in which  $p_{ij}$  can, very naturally, be interpreted as the probability that a request has community (city)  $i$  as origin and  $j$  as destination. For a request from city  $i$  to city  $j$ , the requested departure times has instead been sampled uniformly in the interval  $[T_0, T_0 + T - t_{ij}]$  (notation refers to the previous section). More succinctly,  $\tau_i \sim \mathcal{U}([T_0, T_0 + T - t_{ij}])$  (note that the single requested departure times are uniformly distributed, but if we consider them jointly, this is no more the case because of the "time of the end of the shift" constraint). The limitations on the interval have been made in order to assure that each request was admissible in the prescribed time slot. Obviously, this is also a very unrealistic

assumption, given that it is reasonable to think that most of the requests are early in the morning, when everyone has to get to their workplace or at school or – help them – to university. The requests have been saved in the file `requests.txt`. In figure 1.2 we represent all municipalities in the province of Cosenza weighted by how many times they appear as origin or destination in the `requests.txt` file, created with the proposed model:



**Figure 1.2:** Municipalities in the province of Cosenza weighted by demand in the described way. The bigger the red circle, the bigger the demand.

## Chapter 2: Algorithm

### 2.1 Description

Recalling what has been said in the problem presentation, the approach described in [3] – which we now propose, casting it in our context – seems reasonable. The idea is fairly simple: the problem of finding a feasible set of routes which is also close to "the best" in the sense of the objective function described above can be split into two parts.

- The first part, called the **insertion** algorithm, is concerned with the feasibility of the requests. This should be a constructive, heuristic algorithm whose job is to spit out the best possible solution given the new arriving request (whether it is "truly arriving", as in an online mode, or "fakely arriving", as in an offline mode) without invalidating the feasibility of the previously gathered ones. This first part follows essentially a "greedy" strategy, in which for each new incoming request we only check whether it is possible to accept it and construct the associated set of routes which is optimal with respect to this single, incoming request. This obviously depends on the arrival order and comes with the drawback that the found solution might not be optimal as far as all the requests in their entirety are concerned.
- The second part, which we might call the **optimization** algorithm, is concerned with the increase in quality of the found solution. In the online setting, it operates between a request and the next incoming one, via reshuffling: by mixing the arrival order of the requests and applying again the insertion algorithm of the first part, its job is to minimize or at least decrease the drawback of the greedy strategy by selecting the set of routes for which the current batch of requests (including the incoming one) is feasible yielding the smallest possible value of the objective function. Precisely, it reshuffles only the active requests, i.e. those request which have not yet been completed. In the offline setting, in which the requests are known in advance, all of them are reshuffled. Clearly, because of computational infeasibility not all possible reshuffling are investigated. The online setting comes with a complication: indeed, when optimizing we must ensure that requests that have been previously accepted do not get rejected in the optimized version of the solution (since we assume that if a request is considered feasible, this is immediately told to the customer).

An important thing to stress is that the found solution is not optimal. Indeed, in the second part we do not check for all possible reshufflings: by greedily evaluating requests that reach the system we might be accepting some of them which lead to the refusal of several "easier" ones, and we never come back to take the "difficult" requests away if we already accepted them!

In the pseudocode and in the implementation of the algorithm, we will focus on the offline setting, but the code is easily adaptable for the online one. In particular, we will discuss a datastructure which could be useful in a web based architecture dealing with this problem.

### 2.2 Pseudocode and comments

We now formalize what we said in the previous section by writing down the pseudocode of the algorithm. We will first state the constructive insertion heuristic, i.e. the first part, and then the second part: Let  $z(T)$  denote the total customer delay – the objective function – of the requests served by the set of routes  $T$ , and  $\Delta z_p$  is the increase in the objective caused by an insertion. The function  $\text{FEASIBLE}(T, k)$  returns `true` if and only

if the tour  $T_k$ , operated by vehicle  $k$ , satisfies all the feasibility requirements stated before: the pick-up time windows  $[\tau_i, \tau_i + W]$ , the maximum ride times  $t_{i,n+i} + \vartheta$ , the capacity  $Q$ , and the return to the depot  $\rho(k)$  within the shift  $[T_0, T_0 + T]$ . We also report the pseudocode for it.

---

**Algorithm 1** FEASIBLE
 

---

**Require:** a tour  $T'$  operated by vehicle  $k \in K$  and the request  $\rho$ ; the parameters  $Q, W, \vartheta, d, T_0, T$ , the loads  $q_i$ , the requests  $\tau_i$ , the travel times  $t_{ij}$  and the set of pick-up nodes  $P$ .

```

1:  $t \leftarrow T_0$ ;  $\text{LOAD} \leftarrow 0$ ;  $c \leftarrow o_k$ ;  $d' \leftarrow 0$            ▷ starting time, empty vehicle at its depot, 0 service time
2: for stop  $\nu$  of  $T'$  do
3:    $t \leftarrow t + d' + t_{c,\nu}$                                      ▷ reach  $\nu$  from the current stop
4:   if  $\nu \in P$  then                                             ▷ the next stop is a pick-up point
5:      $t \leftarrow \max(t, \tau_\nu)$                                    ▷ wait if the vehicle's early
6:     if  $t > \tau_\nu + W$  then                                       ▷ out of the pick-up window
7:       return FALSE
8:     end if
9:     if  $\text{LOAD} + q_\nu > Q$  then                                     ▷ capacity exceeded
10:      return FALSE
11:    end if
12:     $\text{LOAD} \leftarrow \text{LOAD} + q_\nu$                                    ▷ the vehicle picks the passengers up
13:     $S_\nu \leftarrow t$                                              ▷ store the pick-up service start time
14:  else                                                         ▷ the next stop is a destination
15:     $i \leftarrow \nu - n$                                            ▷ departure associated to current destination
16:    if  $t - (S_i + d) > t_{i,\nu} + \vartheta$  then                         ▷ maximum ride time of user  $i$  exceeded
17:      return FALSE
18:    end if
19:     $\text{LOAD} \leftarrow \text{LOAD} - q_i$                                    ▷ the vehicle delivers the passengers
20:  end if
21:   $c \leftarrow \nu$ ;  $d' \leftarrow d$ 
22: end for
23: if  $t + d' + t_{c,o'_k} > T_0 + T$  then                         ▷ return to the depot not within the shift
24:  return FALSE
25: end if
26: return TRUE

```

---

**Algorithm 2** INSERTION

**Require:** the set of requests  $R = (\varrho_1, \dots, \varrho_{|R|})$ , where each request is a triplet  $(i, n + i, \tau_i)$ ; the set of vehicles  $K$  with the set of associated depots  $(\rho(k))_{k=1}^K$ ; the parameters  $Q, W, \vartheta, d, T_0, T$ .

```

1:  $T_k \leftarrow \emptyset \forall k \in K; \mathcal{A} \leftarrow \emptyset; \mathcal{R} \leftarrow \emptyset$   $\triangleright$  K empty tours, empty sets of accepted and rejected requests
2: for  $\varrho = (i, n + i, \tau_i) \in R$  do
3:    $\Delta_\varrho^* \leftarrow +\infty; k_\varrho^* \leftarrow \text{NONE}; T_\varrho^* \leftarrow \text{NONE}$ 
4:   for  $k \in K$  do
5:     for  $(p, q) \in \mathbb{Z}^2 : 0 \leq p \leq q \leq |T_k|$  do  $\triangleright (p, q)$  are the inserting positions
6:        $T' \leftarrow T_k$  with  $i$  inserted at  $p$  and  $n + i$  inserted at  $q$ 
7:       if FEASIBLE( $T', k, \varrho$ ) then
8:          $\Delta z_\varrho \leftarrow z(T') - z(T_k)$ 
9:         if  $\Delta z_\varrho < \Delta_\varrho^*$  then  $\triangleright$  cheapest feasible insertion so far
10:           $\Delta_\varrho^* \leftarrow \Delta z_\varrho; k_\varrho^* \leftarrow k; T_\varrho^* \leftarrow T'$ 
11:        end if
12:      end if
13:    end for
14:  end for
15:  if  $k_\varrho^* = \text{NONE}$  then  $\triangleright$  no feasible insertion exists
16:     $\mathcal{R} \leftarrow \mathcal{R} \cup \{\varrho\}$   $\triangleright$  reject
17:  else
18:     $T_{k_\varrho^*} \leftarrow T_\varrho^*; \mathcal{A} \leftarrow \mathcal{A} \cup \{\varrho\}$   $\triangleright$  accept
19:  end if
20: end for
21: return  $(\{T_k\}_{k \in K}, \mathcal{A}, \mathcal{R})$ 

```

**Algorithm 3** OPTIMIZATION

**Require:** the set of requests  $R$ ; the number of reshufflings which one is willing to explore  $N$  (connected to the idle time between the arrival of a request and the following one)

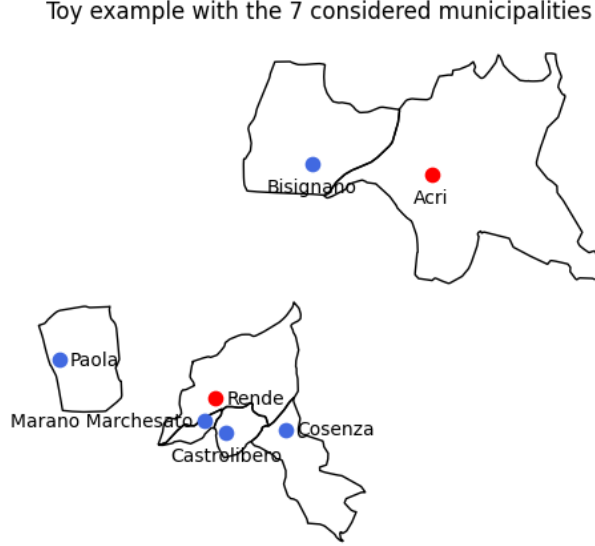
```

1:  $(\{T_k^*\}, \mathcal{A}^*, \cdot) \leftarrow \text{INSERTION}(R, \dots); z^* \leftarrow \sum_{k \in K} z(T_k^*)$ 
2: for  $i = 1, \dots, N$  do  $\triangleright$  second level
3:    $R_\sigma \leftarrow$  random permutation of the requests
4:    $(\{T_k\}, \mathcal{A}, \cdot) \leftarrow \text{INSERTION}(R_\sigma, \dots); z \leftarrow \sum_{k \in K} z(T_k)$ 
5:   if  $(|\mathcal{A}| > |\mathcal{A}^*|)$  or  $(|\mathcal{A}| = |\mathcal{A}^*| \text{ and } z < z^*)$  then  $\triangleright$  more served, then less delay
6:      $\{T_k^*\} \leftarrow \{T_k\}; \mathcal{A}^* \leftarrow \mathcal{A}; z^* \leftarrow z$ 
7:   end if
8: end for
9: return  $(\{T_k^*\}, \mathcal{A}^*)$ 

```

## 2.3 A toy example

We now explore a toy example so as to get a concrete grasp of how the algorithm works. We will consider seven municipalities: Rende, Marano Marchesato, Castrolibero, Cosenza, Paola, Bisignano and Acri. We suppose that there are  $K = 2$  vehicles, one of which is based in Acri and the other one in Rende, i.e.  $\mathcal{D} = \{\text{Rende}, \text{Acri}\}$ ,  $\rho(1) = \text{Rende}$ ,  $\rho(2) = \text{Acri}$ .



**Figure 2.1:** Plot representing the situation described in the toy example. Depots are in red.

The (symmetric, in minutes, approximated so as not to have decimals) travel times  $t_{ij}$  are taken from the file `distance-duration_table.txt` and reported in table 2.1.

|                   | Rende | Marano Marchesato | Castrolibero | Cosenza | Paola | Bisignano | Acri |
|-------------------|-------|-------------------|--------------|---------|-------|-----------|------|
| Rende             | 0     | 8                 | 13           | 11      | 23    | 33        | 48   |
| Marano Marchesato | 8     | 0                 | 10           | 15      | 26    | 38        | 53   |
| Castrolibero      | 13    | 10                | 0            | 11      | 33    | 39        | 54   |
| Cosenza           | 11    | 15                | 11           | 0       | 33    | 31        | 46   |
| Paola             | 23    | 26                | 33           | 33      | 0     | 51        | 66   |
| Bisignano         | 33    | 38                | 39           | 31      | 51    | 0         | 21   |
| Acri              | 48    | 53                | 54           | 46      | 66    | 21        | 0    |

**Table 2.1:** Travel times  $t_{ij}$  in minutes among the seven municipalities of this toy example.

Furthermore, we consider four requests. Each is a triple  $\varrho = (i, n + i, \tau_i)$  consisting of origin, destination, requested departure time. These are listed in table 2.2.

|             | origin            | destination  | $\tau$ |
|-------------|-------------------|--------------|--------|
| $\varrho_1$ | Cosenza           | Paola        | 7:00   |
| $\varrho_2$ | Marano Marchesato | Castrolibero | 7:05   |
| $\varrho_3$ | Cosenza           | Castrolibero | 7:05   |
| $\varrho_4$ | Bisignano         | Acri         | 7:20   |

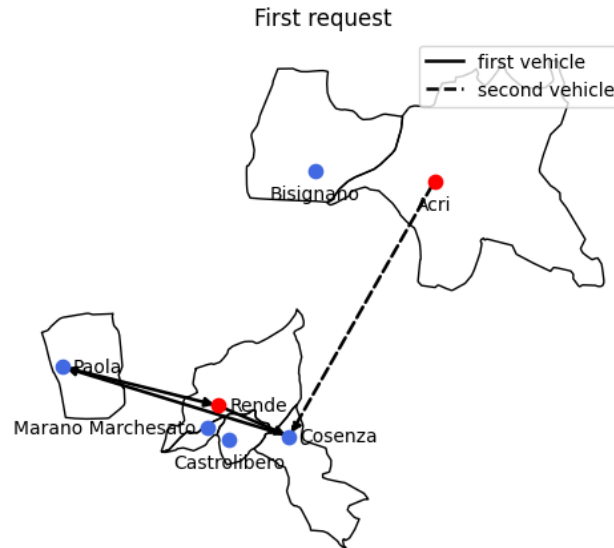
**Table 2.2:** The four requests, in their order of arrival.

We suppose that the fleet is homogeneous with capacity  $Q = 2$ , that the pick-up tolerance is not specified by the customer (a simplification we have already done in the discussion of the model for our case) and is constant, i.e.  $W = 20\text{min}$ , that the ride-time tolerance is constant, i.e.  $\vartheta = 20\text{min}$ , that the service time is negligible ( $d = 0$ ), and that every vehicle operates within the common shift  $[T_0, T_0 + T] = [7:00, 8:10]$ , i.e.  $T_0 = 420$  and  $T = 70\text{min}$ . Finally, we also suppose that each request carries a single passenger ( $q_i = 1$ ).

Following the flow described in the pseudocode, we will now consider the execution of the insertion algorithm only, as well as the one of the entire algorithm (insertion + optimization, i.e. insertion + reshuffling).

- **Insertion only.** The four requests are passed to the insertion algorithm, together with all the constants described above.

1.  $\varrho_1$  is considered first, since it is the first request.



**Figure 2.2:** Schema of the first request flow.

The first vehicle is considered, i.e. the one based in Rende. Since at the beginning the tour is empty, the only possible insertion is pick-up at Cosenza followed by delivery at Paola, so that, symbolically,  $T' = [\text{Cosenza}, \text{Paola}]$ . The feasibility of this insertion is assessed. Leaving Rende at  $T_0 = 7:00$ , the vehicle reaches Cosenza after  $t_{\text{Rende}, \text{Cosenza}} = 11\text{min}$ , i.e. at 7:11, which falls inside the pick-up window  $[\tau_1, \tau_1 + W] = [7:00, 7:20]$ . Being empty, the capacity  $Q = 2$  is not exceeded if one passenger gets on the bus. The vehicle then reaches Paola at  $7:11 + 33\text{min} = 7:44$ . In this case, the ride lasts exactly the direct time, so the ride-time bound  $t_{\text{Cosenza}, \text{Paola}} + \vartheta$  is met and the evaluation on the return time to Rende yields a feasible result:  $7:44 + 23\text{min} = 8:07 \leq T_0 + T = 8:10$ . The

insertion is therefore considered feasible, and the cost is

$$z = \underbrace{7:11 - 7:00}_{\text{pick-up delay, 11min}} + \underbrace{7:44 - 7:00 - 33\text{min}}_{\text{arrival delay, 11min}} = 22\text{min}.$$

The second vehicle from Acri is tried as well, but reaching Cosenza from Acri takes 46min, thus the arrival would be at 7:46, beyond the window upper bound at 7:20, thus the tour would be considered infeasible. Hence  $\varrho_1$  is accepted by the Rende vehicle, whose tour becomes  $T_{\text{Rende}} = [\text{Cosenza}, \text{Paola}]$ .

2.  $\varrho_2$  is now considered. For Rende's vehicle, there are

$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{2 + 2}{2} = \binom{4}{2} = 6$$

possible insertions:

| $(p, q)$ | candidate tour $T'$                                                         |
|----------|-----------------------------------------------------------------------------|
| (0, 0)   | Marano $\rightarrow$ Castrolibero $\rightarrow$ Cosenza $\rightarrow$ Paola |
| (0, 1)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (0, 2)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (1, 1)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (1, 2)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (2, 2)   | Cosenza $\rightarrow$ Paola $\rightarrow$ Marano $\rightarrow$ Castrolibero |

**Table 2.3:** Candidate insertions of  $\varrho_2$ .

"Marano" stands for "Marano Marchesato".

- (a). The first one gets rejected because the travel Rende (depot)  $\rightarrow$  Marano Marchesato  $\rightarrow$  Castrolibero  $\rightarrow$  Cosenza takes  $(8 + 10 + 11)\text{min} = 29\text{min}$ , and departing from Marano Marchesato at 7:08 means that the vehicle would arrive at Cosenza at 7:29, 9 minutes late for the pick-up of the customer of  $\varrho_1$ .

First candidate, Rende's vehicle



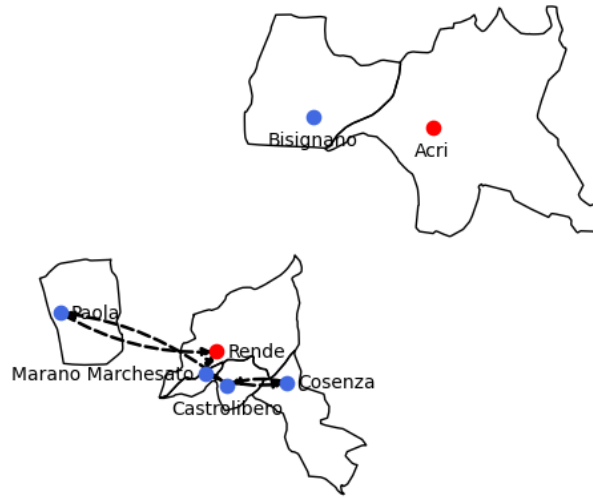
**Figure 2.3:** Schema of the first candidate for Rende's vehicle for the second request.

- (b). The second one gets rejected because the travel Rende (depot)  $\rightarrow$  Marano Marchesato  $\rightarrow$  Cosenza



takes  $(8 + 15)$  min, and departing from Marano Marchesato at 7:08 means that the vehicle would arrive at Cosenza at 7:23, 3 minutes late for the pick-up of the customer of  $\varrho_1$ .

Second candidate, Rende's vehicle



**Figure 2.4:** Schema of the second candidate for Rende's vehicle for the second request.

- (c). The third one gets rejected for the same reason (there is again the route Rende (depot)  $\rightarrow$  Marano Marchesato  $\rightarrow$  Cosenza).

Third candidate, Rende's vehicle



**Figure 2.5:** Schema of the third candidate for Rende's vehicle for the second request.

- (d). The fourth one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Marano Marchesato takes  $(11 + 15)$  min, and departing from Cosenza at 7:11 after having picked up the customer of  $\varrho_1$  means that the vehicle would arrive at Marano Marchesato at 7:26, 1 minute late for the pick-up of the customer of  $\varrho_2$ .

Fourth candidate, Rende's vehicle

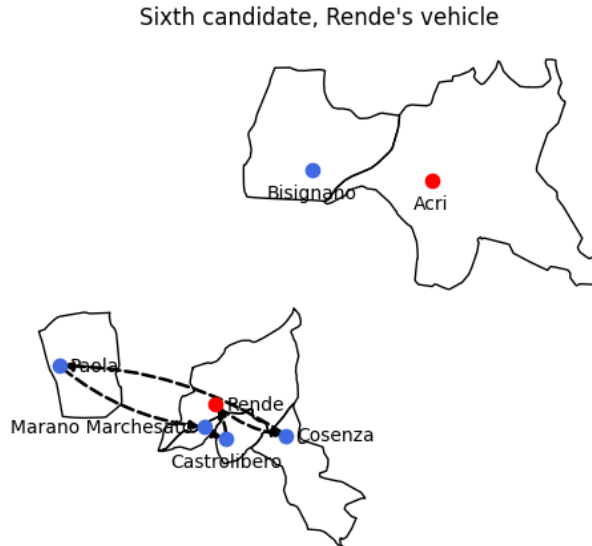
**Figure 2.6:** Schema of the fourth candidate for Rende's vehicle for the second request.

- (e). The fifth one gets rejected for the same reason (there is again the route Rende (depot)→Cosenza→Marano Marchesato).

Fifth candidate, Rende's vehicle

**Figure 2.7:** Schema of the fifth candidate for Rende's vehicle for the second request.

- (f). The sixth one gets rejected because the travel Rende (depot)→Cosenza→Paola→Marano Marchesato takes  $(11 + 33 + 26)$ min, meaning that the vehicle, departing from Rende at 7 : 00 and serving the customer of  $q_1$ , would get back to Marano Marchesato only at 8:10, 45 minutes late for the pick-up of the customer of  $q_2$ .



**Figure 2.8:** Schema of the sixth candidate for Rende's vehicle for the second request.

For Acri's vehicle, which is still empty, only one insertion has to be considered. Yet  $t_{\text{Acri,Marano}} = 53\text{min}$ , so its earliest pick-up could be at 7:53 outside the available window of the customer of  $q_2$ , [7:05, 7:25].



**Figure 2.9:** Schema of the first (and only) candidate for Acri's vehicle for the second request.

With no feasible insertion on either vehicle,  $q_2$  is rejected.

3.  $q_3$  is now considered. For Rende's vehicle, there are again

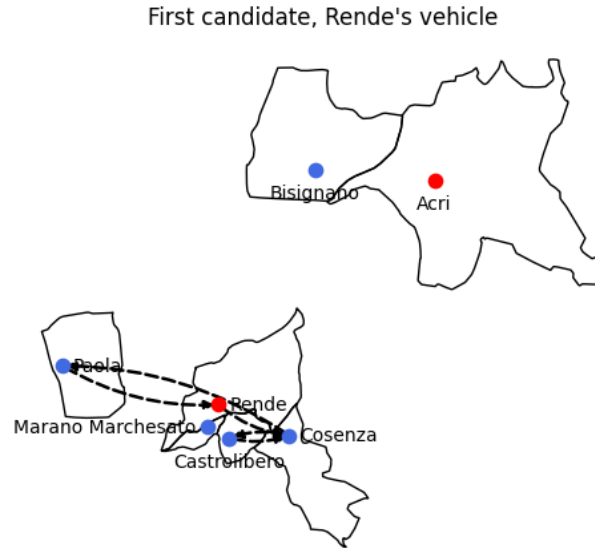
$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{2 + 2}{2} = \binom{4}{2} = 6$$

possible insertions:

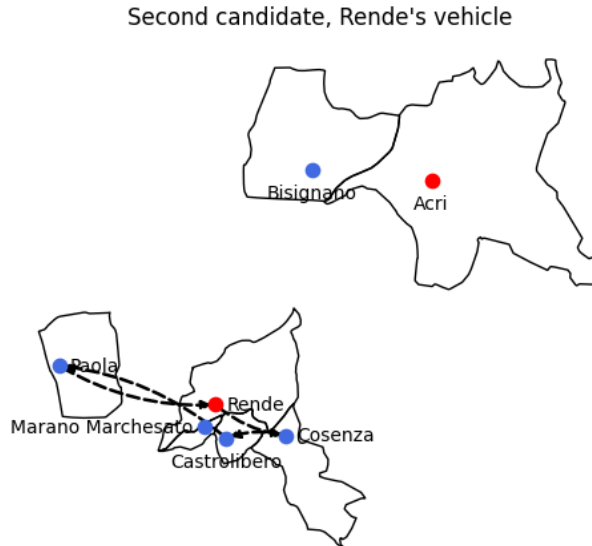
| $(p, q)$ | candidate tour $T'$                                                          |
|----------|------------------------------------------------------------------------------|
| (0, 0)   | Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Cosenza $\rightarrow$ Paola |
| (0, 1)   | Cosenza $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (0, 2)   | Cosenza $\rightarrow$ Cosenza $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (1, 1)   | Cosenza $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (1, 2)   | Cosenza $\rightarrow$ Cosenza $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (2, 2)   | Cosenza $\rightarrow$ Paola $\rightarrow$ Cosenza $\rightarrow$ Castrolibero |

**Table 2.4:** Candidate insertions of  $\varrho_3$ .

- (a). The first one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Castrolibero  $\rightarrow$  Cosenza takes  $(11 + 11 + 11)\text{min} = 33\text{min}$ , and departing from Rende at 7:00 means that the vehicle would arrive at Cosenza (for the second time) at 7:33, 13 minutes late for the pick-up of the customer of  $\varrho_1$ .

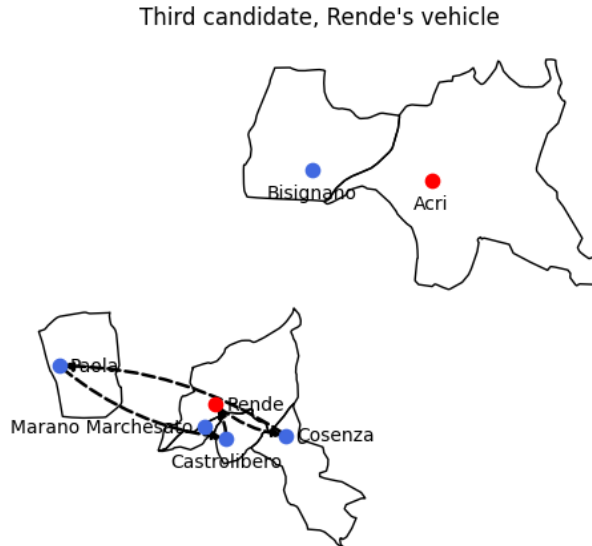
**Figure 2.10:** Schema of the first candidate for Rende's vehicle for the third request.

- (b). The second one gets rejected because, picking up the customers of  $\varrho_3$  and  $\varrho_1$  together at Cosenza at 7:11, delivering the customer of  $\varrho_3$  at Castrolibero at  $7:11 + 11\text{min} = 7:22$  and the customer of  $\varrho_1$  at Paola ( $7:22 + 33\text{min} = 7:55$ ), the vehicle would get back to Rende only at  $7:55 + 23 = 8:18$ , 8 minutes after the end of the shift  $T_0 + T = 8:10$ .



**Figure 2.11:** Schema of the second candidate for Rende's vehicle for the third request.

- (c). The third one gets rejected because, picking up the customers of  $\varrho_3$  and  $\varrho_1$  together at Cosenza at 7:11, delivering the customer of  $\varrho_1$  at Paola at  $7:11 + 33\text{min} = 7:44$ , the customer of  $\varrho_3$  would be delivered at Castrolibero only at  $7:44 + 33\text{min} = 8:17$ , yielding both a ride of 66min against the maximum ride time  $t_{\text{Cosenza,Castrolibero}} + \vartheta = (11 + 20)\text{min} = 31\text{min}$  and also a delay after the end of the shift at 8:10.



**Figure 2.12:** Schema of the third candidate for Rende's vehicle for the third request.

- (d). The fourth one gets rejected for the same reason as the second one.

Fourth candidate, Rende's vehicle

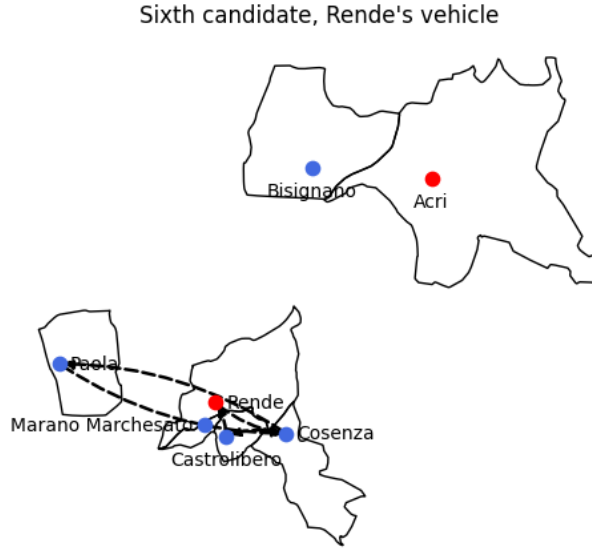
**Figure 2.13:** Schema of the fourth candidate for Rende's vehicle for the third request.

- (e). The fifth one gets rejected for the same reason as the third one.

Fifth candidate, Rende's vehicle

**Figure 2.14:** Schema of the fifth candidate for Rende's vehicle for the third request.

- (f). The sixth one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Paola  $\rightarrow$  Cosenza takes  $(11 + 33 + 33)\text{min} = 77\text{min}$ , meaning that the vehicle, departing from Rende at 7:00, delivering the customer of  $q_1$ , would come back to Cosenza only at 8:17, 52 minutes late for the pick-up of the customer of  $q_3$ .



**Figure 2.15:** Schema of the sixth candidate for Rende's vehicle for the third request.

For Acri's vehicle, which is still empty, only one insertion has to be considered. Yet  $t_{\text{Acri}, \text{Cosenza}} = 46\text{min}$ , so its earliest pick-up could be at 7:46 outside the available window of the customer of  $\varrho_3$ , [7:05, 7:25].



**Figure 2.16:** Schema of the first (and only) candidate for Acri's vehicle for the third request.

With no feasible insertion on either vehicle,  $\varrho_3$  is rejected.

4.  $\varrho_4$  is now considered. For Rende's vehicle, there are again

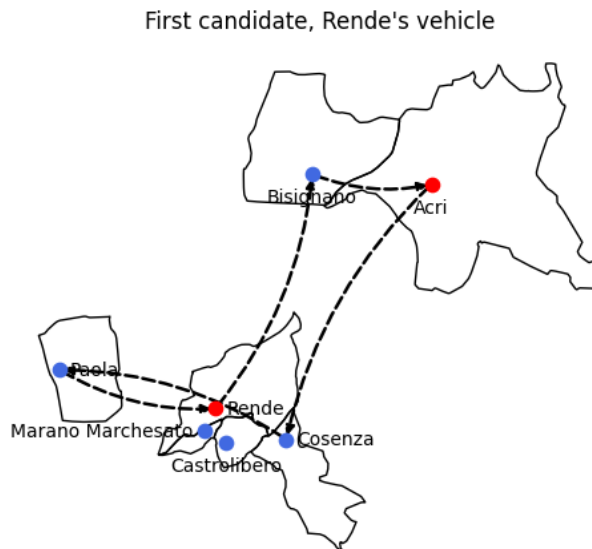
$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{2 + 2}{2} = \binom{4}{2} = 6$$

possible insertions:

| $(p, q)$ | candidate tour $T'$                |
|----------|------------------------------------|
| (0, 0)   | Bisignano → Acri → Cosenza → Paola |
| (0, 1)   | Bisignano → Cosenza → Acri → Paola |
| (0, 2)   | Bisignano → Cosenza → Paola → Acri |
| (1, 1)   | Cosenza → Bisignano → Acri → Paola |
| (1, 2)   | Cosenza → Bisignano → Paola → Acri |
| (2, 2)   | Cosenza → Paola → Bisignano → Acri |

**Table 2.5:** Candidate insertions of  $q_4$ .

- (a). The first one gets rejected because the travel Rende (depot)→Bisignano→Acri→Cosenza takes  $(33 + 21 + 46)$ min, and departing from Rende at 7:00 means that the vehicle would arrive at Cosenza at  $7:00 + 100\text{min} = 8:40$ , 80 minutes late for the pick-up of the customer of  $q_1$  and over the end of the shift time, 8:10.



**Figure 2.17:** Schema of the first candidate for Rende's vehicle for the fourth request.

- (b). The second one gets rejected because the travel Rende (depot)→Bisignano→Cosenza takes  $(33 + 31)$ min, and departing from Rende at 7:00 means that the vehicle would arrive at Cosenza at  $7:00 + 64\text{min} = 8:04$ , 44 minutes late for the pick-up of the customer of  $q_1$ .



Second candidate, Rende's vehicle

**Figure 2.18:** Schema of the second candidate for Rende's vehicle for the fourth request.

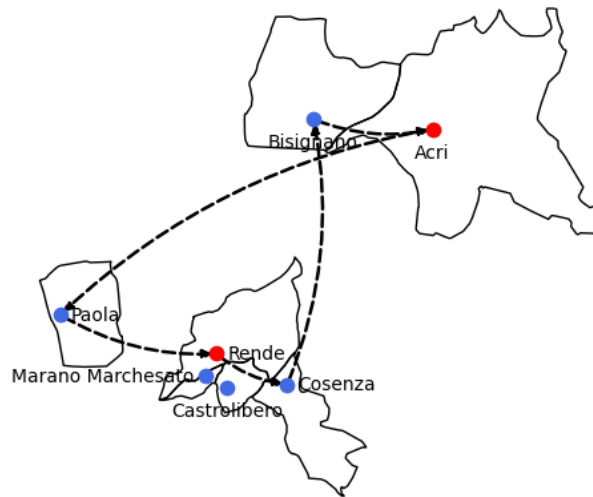
- (c). The third one gets rejected for the same reason (there is again the route Rende (depot)→Bisignano→Cosenza).

Third candidate, Rende's vehicle

**Figure 2.19:** Schema of the third candidate for Rende's vehicle for the fourth request.

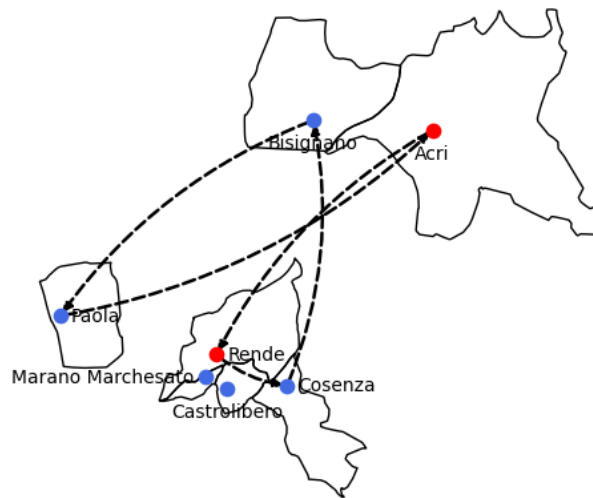
- (d). The fourth one gets rejected because the travel Rende (depot)→Cosenza→Bisignano takes  $(11 + 31)\text{min}$ , and departing from Cosenza at 7:11 after having picked up the customer of  $\varrho_1$  means that the vehicle would arrive at Bisignano at  $7:11 + 31\text{min} = 7:42$ , 2 minutes late for the pick-up of the customer of  $\varrho_4$ .

Fourth candidate, Rende's vehicle

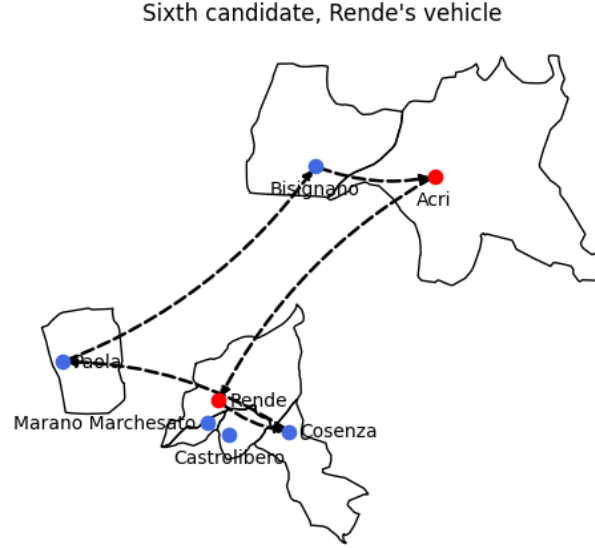
**Figure 2.20:** Schema of the fourth candidate for Rende's vehicle for the fourth request.

- (e). The fifth one gets rejected for the same reason (there is again the route Rende (depot)→Cosenza→Bisignano).

Fifth candidate, Rende's vehicle

**Figure 2.21:** Schema of the fifth candidate for Rende's vehicle for the fourth request.

- (f). The sixth one gets rejected because the travel Rende (depot)→Cosenza→Paola→Bisignano takes  $(11 + 33 + 51)\text{min} = 95\text{min}$ , meaning that the vehicle, departing from Rende at 7:00 and serving the customer of  $\varrho_1$ , would arrive at Bisignano only at 8:35, 55 minutes late for the pick-up of the customer of  $\varrho_4$  and well beyond the end of the shift at 8:10.

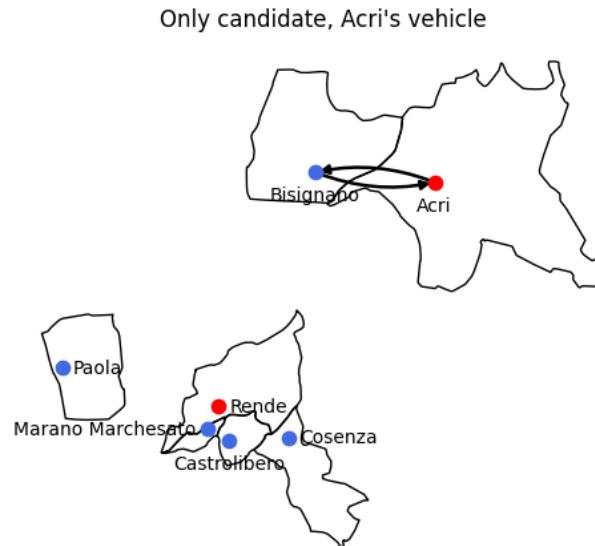


**Figure 2.22:** Schema of the sixth candidate for Rende's vehicle for the fourth request.

For Acri's vehicle, which is still empty, only one insertion has to be considered: Bisignano→Acri. Leaving Acri at 7:00, the vehicle reaches Bisignano after  $t_{\text{Acri},\text{Bisignano}} = 21\text{min}$ , with pick-up at 7:21, in time for the pick-up window of the customer of  $\varrho_4$ ,  $[7:20, 7:40]$ , and delivers them at Acri after  $t_{\text{Bisignano},\text{Acri}} = 21\text{min}$ , at 7:42, where it is already at its depot, within the end of the shift time at 8:10. The insertion is feasible, with cost

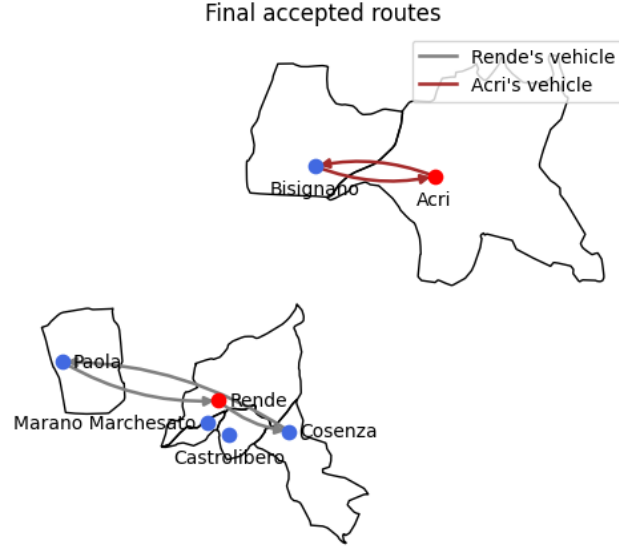
$$z = \underbrace{(7:21 - 7:20)}_{\text{pick-up delay, 1min}} + \underbrace{(7:42 - 7:20 - 21)}_{\text{arrival delay, 1min}} = 2\text{min}.$$

Being the only feasible insertion,  $\varrho_4$  is accepted and assigned to Acri's vehicle, whose tour becomes  $T_{\text{Acri}} = [\text{Bisignano}, \text{Acri}]$ .



**Figure 2.23:** Schema of the first (and only) accepted candidate for Acri's vehicle for the fourth request.

Summing up, insertion alone accepts  $\{\varrho_1, \varrho_4\}$  and rejects  $\{\varrho_2, \varrho_3\}$ , achieving a total cost of  $z = 22 + 2 = 24$  (minutes).



**Figure 2.24:** Schema of the final routes of the two vehicles with the insertion algorithm only.

- **Optimization.** Here we huffle the order of arrival of the requests, run again the insertion algorithm on the shuffled set and find the "best" solution, where "best" means the solution with the greatest number of customers first and the lowest cost (total delay) then. The 24 possible reshufflings are:

$$\begin{aligned}
 &\{\varrho_1, \varrho_2, \varrho_3, \varrho_4\} \quad \{\varrho_1, \varrho_2, \varrho_4, \varrho_3\} \quad \{\varrho_1, \varrho_3, \varrho_2, \varrho_4\} \quad \{\varrho_1, \varrho_3, \varrho_4, \varrho_2\} \quad \{\varrho_1, \varrho_4, \varrho_2, \varrho_3\} \quad \{\varrho_1, \varrho_4, \varrho_3, \varrho_2\} \\
 &\{\varrho_2, \varrho_1, \varrho_3, \varrho_4\} \quad \{\varrho_2, \varrho_1, \varrho_4, \varrho_3\} \quad \{\varrho_2, \varrho_3, \varrho_1, \varrho_4\} \quad \{\varrho_2, \varrho_3, \varrho_4, \varrho_1\} \quad \{\varrho_2, \varrho_4, \varrho_1, \varrho_3\} \quad \{\varrho_2, \varrho_4, \varrho_3, \varrho_1\} \\
 &\{\varrho_3, \varrho_1, \varrho_2, \varrho_4\} \quad \{\varrho_3, \varrho_1, \varrho_4, \varrho_2\} \quad \{\varrho_3, \varrho_2, \varrho_1, \varrho_4\} \quad \{\varrho_3, \varrho_2, \varrho_4, \varrho_1\} \quad \{\varrho_3, \varrho_4, \varrho_1, \varrho_2\} \quad \{\varrho_3, \varrho_4, \varrho_2, \varrho_1\} \\
 &\{\varrho_4, \varrho_1, \varrho_2, \varrho_3\} \quad \{\varrho_4, \varrho_1, \varrho_3, \varrho_2\} \quad \{\varrho_4, \varrho_2, \varrho_1, \varrho_3\} \quad \{\varrho_4, \varrho_2, \varrho_3, \varrho_1\} \quad \{\varrho_4, \varrho_3, \varrho_1, \varrho_2\} \quad \{\varrho_4, \varrho_3, \varrho_2, \varrho_1\}.
 \end{aligned}$$

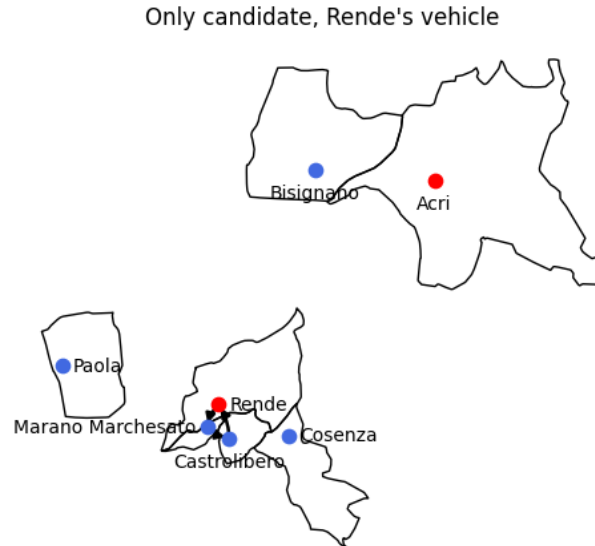
Analyzing each one of the possible 23 reshufflings in the same way in which we did for the original arrival order (which corresponds to the first reshuffling) it is possible to see that we only get two possible solutions:

- the first one is the one we already encountered with the original arrival order, comprising of only two accepted requests, namely  $\{\varrho_1, \varrho_4\}$ , with a total cost of 24min;
- the second one comprises of three accepted requests, namely  $\{\varrho_2, \varrho_3, \varrho_4\}$ , with cost 60min.

As an example, consider the reshuffling:  $\{\varrho_2, \varrho_1, \varrho_3, \varrho_4\}$ . The four requests are fed to the insertion in this order. We now follow the analysis proposed for the insertion algorithm fed with the requests in the original arrival order:

1.  $\varrho_2$  is considered first. The Rende's vehicle is empty, so the only insertion to consider is Marano Marchesato followed by delivery at Castrolibero. Leaving Rende at 7:00, the vehicle reaches Marano after  $t_{\text{Rende, Marano}} = 8\text{min}$ , at  $7:00 + 8\text{min} = 7:08$ , within the pick-up window of the customer of  $\varrho_2$ ,  $[7:05, 7:25]$ . Then it reaches Castrolibero after  $t_{\text{Marano, Castrolibero}} = 10\text{min}$ , at 7:18, and comes back to Rende after  $t_{\text{Castrolibero, Rende}} = 13\text{min}$  at  $7:18 + 13\text{min} = 7:31$ , within the time of the end of the shift, 8:10. The insertion is thus considered feasible, and has cost:

$$z = \underbrace{7:08 - 7:05}_{\text{pick-up delay, 3min}} + \underbrace{7:18 - 7:05 - 10}_{\text{arrival delay, 3min}} = (3 + 3)\text{min} = 6\text{min}.$$



**Figure 2.25:** Schema of the first (and only) candidate for Rende's vehicle for the first request.

The second vehicle from Acri is tried as well, but reaching Marano Marchesato from Acri takes 53min, thus the arrival would be at 7:53, beyond the window upper bound at 7:25, thus the tour would be considered infeasible. Hence  $\varrho_2$  is accepted by Rende's vehicle whose tour becomes  $T_{\text{Rende}} = [\text{Marano Marchesato}, \text{Castrolibero}]$ .



**Figure 2.26:** Schema of the first (and only) candidate for Acri's vehicle for the first request.

2.  $\varrho_1$  is now considered. For Rende's vehicle there are again

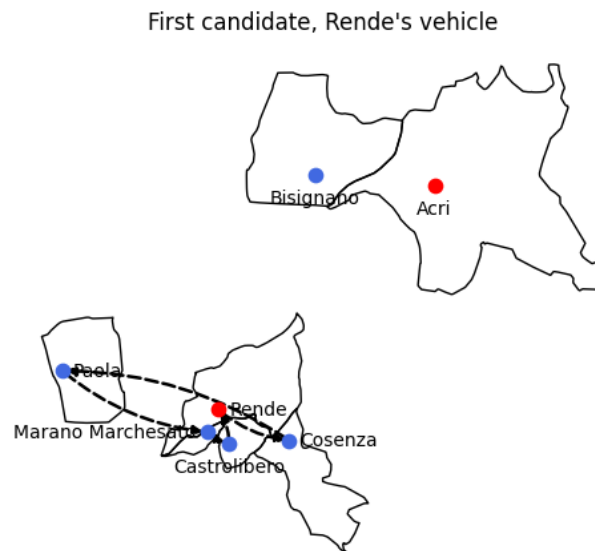
$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{2 + 2}{2} = \binom{4}{2} = 6$$

possible insertions:

| $(p, q)$ | candidate tour $T'$                                                         |
|----------|-----------------------------------------------------------------------------|
| (0, 0)   | Cosenza $\rightarrow$ Paola $\rightarrow$ Marano $\rightarrow$ Castrolibero |
| (0, 1)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (0, 2)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (1, 1)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Paola $\rightarrow$ Castrolibero |
| (1, 2)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Paola |
| (2, 2)   | Marano $\rightarrow$ Castrolibero $\rightarrow$ Cosenza $\rightarrow$ Paola |

**Table 2.6:** Candidate insertions of  $\varrho_1$ .

- (a). The first one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Paola  $\rightarrow$  Marano Marchesato takes  $(11 + 33 + 26)\text{min} = 70\text{min}$ . This means that after delivering the customer of  $\varrho_1$ , the vehicle would reach Marano Marchesato only at 8:10, not within  $\varrho_2$ 's tolerance window  $[7:05, 7:25]$ .

**Figure 2.27:** Schema of the first candidate for Rende's vehicle for the second request.

- (b). The second one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Marano takes  $(11 + 15)\text{min}$ . This means that after picking up the customer of  $\varrho_1$  at Cosenza at (7:11), the vehicle would reach Marano Marchesato only at 7:26, not within  $\varrho_2$ 's tolerance window  $[7:05, 7:25]$ .

Second candidate, Rende's vehicle

**Figure 2.28:** Schema of the second candidate for Rende's vehicle for the second request.

- (c). The third one gets rejected for the same reason (there is again the route Rende (depot)→Cosenza→Marano Marchesato).

Third candidate, Rende's vehicle

**Figure 2.29:** Schema of the third candidate for Rende's vehicle for the second request.

- (d). The fourth one gets rejected because the travel Rende (depot)→Marano→Cosenza takes  $(8 + 15)\text{min}$ . This means that after picking up the customer of  $\varrho_2$  at Marano Marchesato, the vehicle would reach Cosenza only at 7:23, not within  $\varrho_1$ 's tolerance window  $[7:00, 7:20]$ .

Fourth candidate, Rende's vehicle

**Figure 2.30:** Schema of the fourth candidate for Rende's vehicle for the second request.

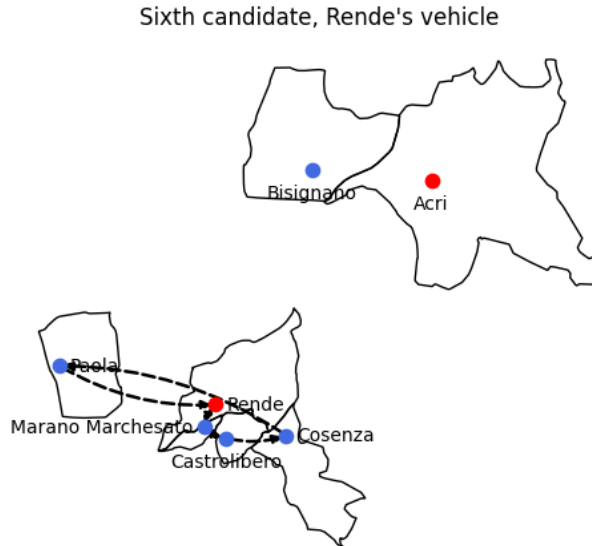
- (e). The fifth one gets rejected for the same reason (there is again the route Rende (depot)→Marano→Cosenza).

Fifth candidate, Rende's vehicle

**Figure 2.31:** Schema of the fifth candidate for Rende's vehicle for the second request.

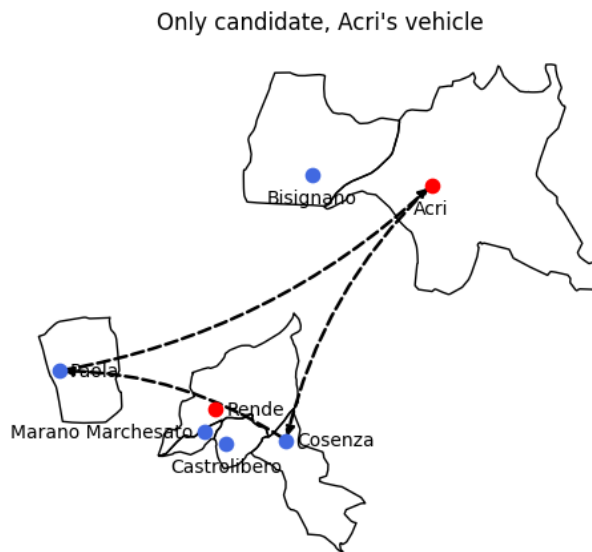
- (f). The sixth one gets rejected because the travel Rende (depot)→Marano→Castrolibero→Cosenza takes  $(8 + 10 + 11)\text{min} = 29\text{min}$ . This means that after picking up  $\varrho_2$  at Marano Marchesato at 7:08 and delivering them at Castrolibero at 7:18, the vehicle would reach Cosenza only at 7:29, not within  $\varrho_1$ 's tolerance window  $[7:00, 7:20]$ .





**Figure 2.32:** Schema of the sixth candidate for Rende's vehicle for the second request.

The Acri vehicle is again infeasible, since  $t_{\text{Acri}, \text{Cosenza}} = 46\text{min}$ , so that the pick-up would be at 7:46, not within  $\varrho_1$ 's tolerance window  $[7:00, 7:20]$ . With no feasible insertion,  $\varrho_1$  is rejected.



**Figure 2.33:** Schema of the first (and only) candidate for Acri's vehicle for the second request.

3.  $\varrho_3$  is now considered. For Rende's vehicle there are again

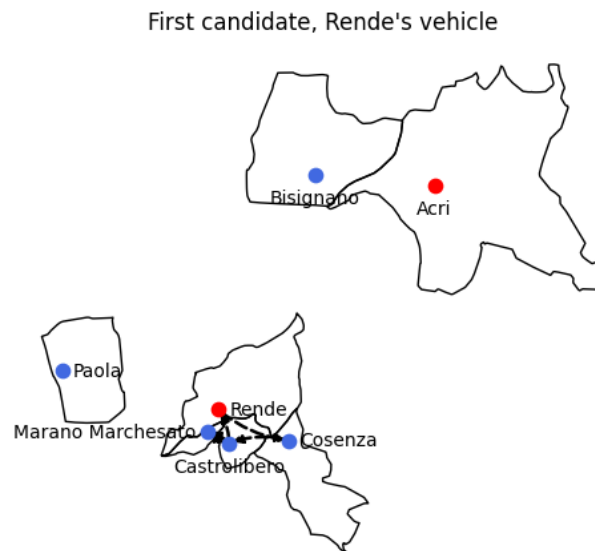
$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{2 + 2}{2} = \binom{4}{2} = 6$$

possible insertions:

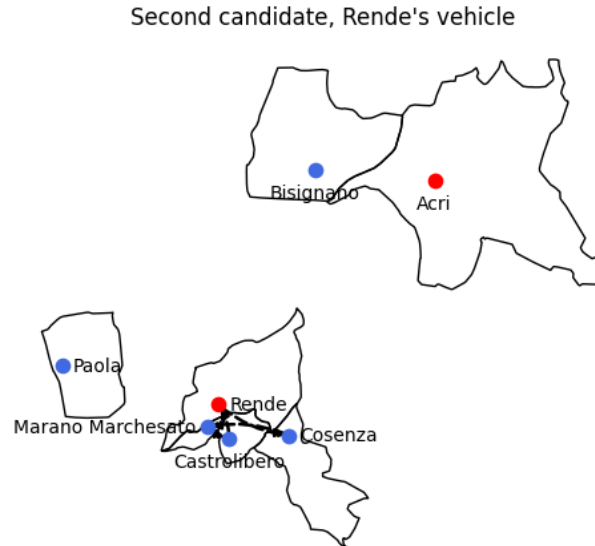
| $(p, q)$ | candidate tour $T'$                                                                |
|----------|------------------------------------------------------------------------------------|
| (0, 0)   | Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Marano $\rightarrow$ Castrolibero |
| (0, 1)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Castrolibero $\rightarrow$ Castrolibero |
| (0, 2)   | Cosenza $\rightarrow$ Marano $\rightarrow$ Castrolibero $\rightarrow$ Castrolibero |
| (1, 1)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Castrolibero |
| (1, 2)   | Marano $\rightarrow$ Cosenza $\rightarrow$ Castrolibero $\rightarrow$ Castrolibero |
| (2, 2)   | Marano $\rightarrow$ Castrolibero $\rightarrow$ Cosenza $\rightarrow$ Castrolibero |

**Table 2.7:** Candidate insertions of  $q_3$ .

- (a). The first one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Castrolibero  $\rightarrow$  Marano Marchesato takes  $(11 + 11 + 10)\text{min} = 32\text{min}$ . This means that after delivering the customer of  $q_3$  at Castrolibero, the vehicle would reach Marano Marchesato only at 7:32, not within  $q_2$ 's tolerance window  $[7:05, 7:25]$ .

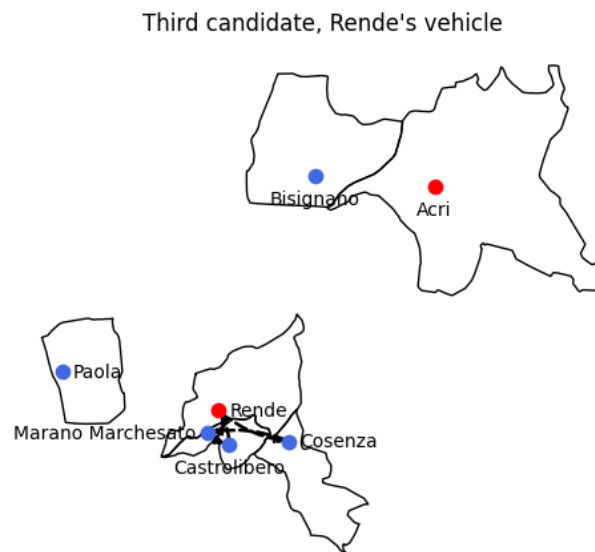
**Figure 2.34:** Schema of the first candidate for Rende's vehicle for the third request.

- (b). The second one gets rejected because the travel Rende (depot)  $\rightarrow$  Cosenza  $\rightarrow$  Marano takes  $(11 + 15)\text{min}$ . This means that after picking up the customer of  $q_3$  at Cosenza at (7:11), the vehicle would reach Marano Marchesato only at 7:26, not within  $q_2$ 's tolerance window  $[7:05, 7:25]$ .



**Figure 2.35:** Schema of the second candidate for Rende's vehicle for the third request.

- (c). The third one gets rejected for the same reason (there is again the route Rende (depot)→Cosenza→Marano Marchesato).



**Figure 2.36:** Schema of the third candidate for Rende's vehicle for the third request.

- (d). The fourth one is feasible: picking up the customer of  $\varrho_2$  at Marano Marchesato at 7:08 and the customer of  $\varrho_3$  at Cosenza at 7:23, so that capacity is met, the vehicle delivers both at Castrolibero at 7:34 and returns to Rende at 7:47. The ride of  $\varrho_3$  lasts 11min, equal to its direct time travel, and that of  $\varrho_2$  lasts 26min, still less than what the delay tolerance allows ( $t_{\text{Marano,Castrolibero}} + \vartheta = (10 + 20)\text{min} = 30\text{min}$ ). The cost of the route rises from 6 to 58min. Indeed, the detour to pick up the customer of  $\varrho_3$  at Cosenza delays the delivery of the customer of  $\varrho_2$  from 7:18 to 7:34, so that

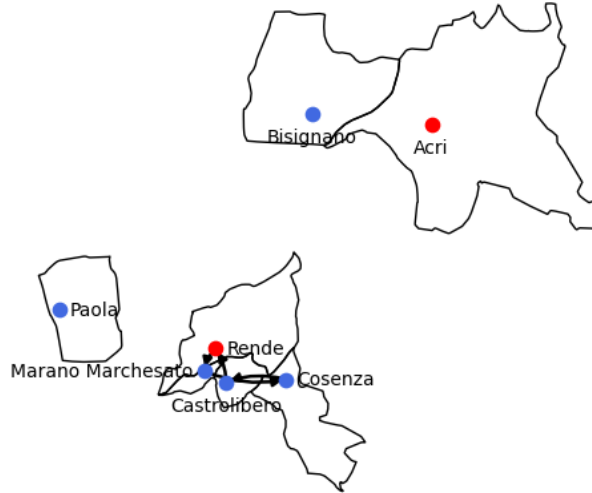
$$\underbrace{(7:08 - 7:05)}_{\text{pick-up delay, 3min}} + \underbrace{(7:34 - 7:05 - 10)}_{\text{arrival delay, 19min}} = 22\text{min},$$

while the newly inserted customer of  $\varrho_3$ , picked up at 7:23 and delivered at 7:34, contributes

$$\underbrace{(7:23 - 7:05)}_{\text{pick-up delay, 18min}} + \underbrace{(7:34 - 7:05 - 11)}_{\text{arrival delay, 18min}} = 36\text{min},$$

for a total cost of  $z = (22 + 36)\text{min} = 58\text{min}$ .

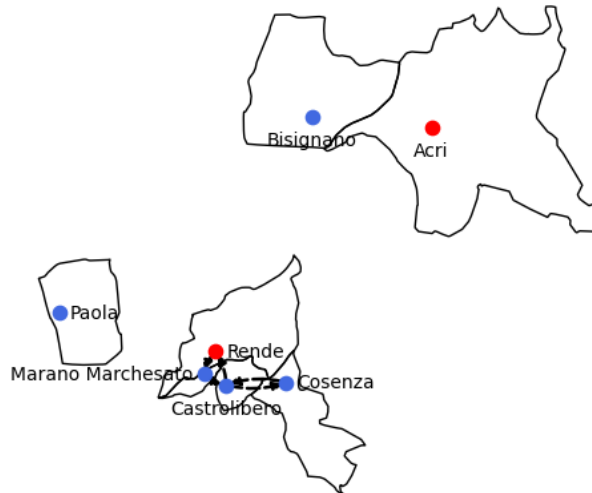
Fourth candidate, Rende's vehicle



**Figure 2.37:** Schema of the fourth candidate for Rende's vehicle for the third request.

- (e). The fifth one is feasible as well, giving the same tour (only the order of the two deliveries at Castrolibero differs).
- (f). The sixth one gets rejected because the travel Rende (depot)  $\rightarrow$  Marano  $\rightarrow$  Castrolibero  $\rightarrow$  Cosenza takes  $(8 + 10 + 11)\text{min} = 29\text{min}$ . This means that after picking up the customer of  $\varrho_2$  at Marano Marchesato at 7:08 and delivering them at Castrolibero at 7:18, the vehicle would reach Cosenza only at 7:29, not within  $\varrho_3$ 's tolerance window  $[7:05, 7:25]$ .

Sixth candidate, Rende's vehicle



**Figure 2.38:** Schema of the sixth candidate for Rende's vehicle for the third request.

The Acri vehicle is again infeasible, since  $t_{\text{Acri}, \text{Cosenza}} = 46\text{min}$ , so that the pick-up would be at 7:46, not within  $\varrho_3$ 's tolerance window  $[7:05, 7:25]$ . The only feasible insertion is therefore the fourth (equivalently the fifth), so  $\varrho_3$  is accepted by Rende's vehicle, sharing it with  $\varrho_2$ :  $T_{\text{Rende}} = [\text{Marano}, \text{Cosenza}, \text{Castrolibero}, \text{Castrolibero}]$ .

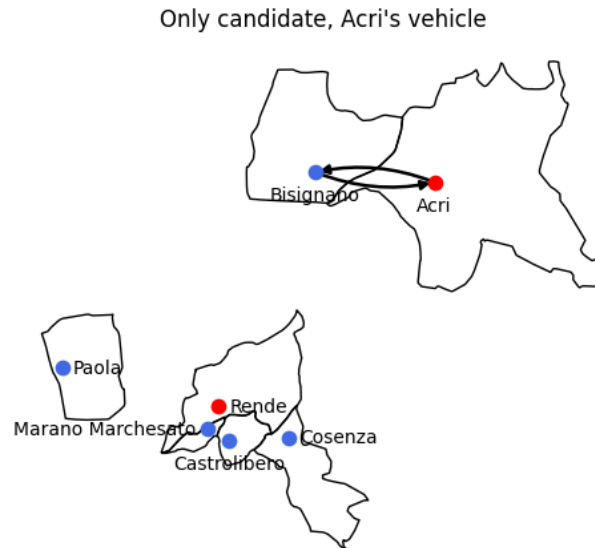


**Figure 2.39:** Schema of the first (and only) candidate for Acri's vehicle for the third request.

4.  $\varrho_4$  is now considered. The Rende vehicle now serves two requests, so that there are:

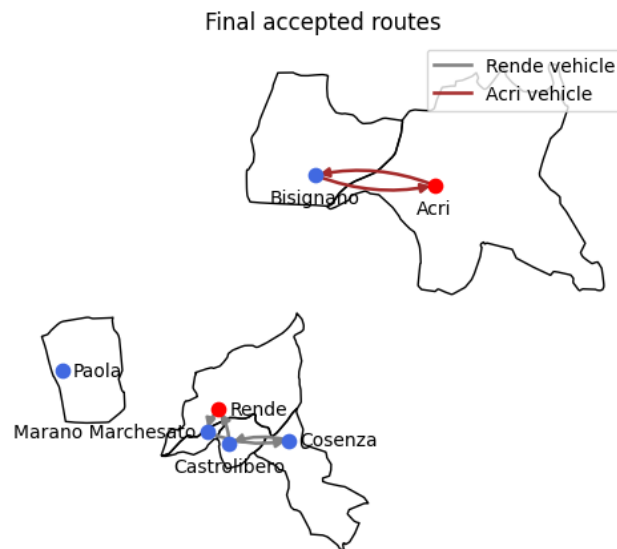
$$\binom{|T_{\text{Rende}}| + 2}{2} = \binom{4 + 2}{2} = \binom{6}{2} = 15$$

candidate insertions. It is clear that they are all infeasible, since Bisignano lies at least 31 minutes from each of Marano Marchesato, Cosenza and Castrolibero and cannot be reached within  $\varrho_4$ 's tolerance window  $[7:20, 7:40]$  without breaking the tolerance windows or ride-times of  $\varrho_2, \varrho_3$  or the end of the shift time. Acri's vehicle, still empty, can serve it directly: Acri  $\rightarrow$  Bisignano takes 21min, picking up the customer of  $\varrho_4$  at 7:21, within their tolerance window  $[7:20, 7:40]$ , and then coming back to Acri (again a 21min travel), with delivery of the customer at 7:42. The cost is  $(7:21 - 7:20) + (7:42 - 7:20 - 21) = 2\text{min}$ . Hence  $\varrho_4$  is accepted by Acri's vehicle.



**Figure 2.40:** Schema of the first (and only) candidate for Acri's vehicle for the fourth request.

Summing up, with this order for the requests the algorithm accepts  $\{\varrho_2, \varrho_3, \varrho_4\}$  and rejects only  $\varrho_1$ , for a total cost of  $z = (58 + 2)\text{min} = 60\text{min}$ . Note that the system is able to accept one request more than the what was able to do with just the insertion on the arrival order.



**Figure 2.41:** Schema of the final routes of the two vehicles with the optimization algorithm.

As a final note, the results in the codes might differ by some decimals. This is due to rounding errors which propagate from the definition of the duration table of the toy example.

## 2.4 Code

In this section we will report the code. In the following one, the presented datastructures and algorithms will be more closely analyzed.

Let us first consider the offline setting, in which all requests are known in advance and we read them from

a .txt file (see section 1.3 about the datasets), namely `requests.txt`, which contains the header:

Source->Destination->Departure

and then all the actual – 1000 – requests in the format the header specifies (more information on the .ipynb file).

We can load them using the following code:

```

1 requests = []
2 with open(path + "requests.txt") as f:
3     next(f) #skip the header
4     rid = 0
5     for line in f:
6         line = line.strip()
7         if not line: # skip empty lines
8             continue
9         origin, destination, departure = line.split("->")
10        h, m = departure.split(":")
11        requests.append((rid, origin, destination, int(h)*60 + int(m)))
12        rid += 1

```

We now report the code to implement the algorithm described above, mirroring as much as possible the pseudocode and the description written for the offline setting:

```

1 def t_ij(i, j):
2     return distance_duration_table[i][j][1] # travel time t_{ij} in minutes between cities i
3         and j
4
5 def evaluate(T_prime, depot, Q, W, THETA, d, T0, T, people):
6     """
7     Basically, one simulation of the T_prime tour operated from "depot". Returns (feasible,
8     z), where feasible is true if the tour does not trespass some of the constraints and
9     z is the total delay of the customers of the tour.
10    """
11    t = T0; load = 0; c = depot; d_prime = 0 # starting time, empty vehicle at the start,
12        service time of the stop, 0 for the depot
13    S = {}; z = 0.0 # pick-up service time beginnings, accumulated delay
14    for kind, nu, req in T_prime: # for each stop in the tour
15        rid, origin, destination, tau = req
16        t = t + d_prime + t_ij(c, nu) # getting to nu from current stop
17        if kind == 'P': # pick-up city
18            t = max(t, tau) # wait if early
19            if t > tau + W: return False, float('inf') # outside pick-up window
20            if load + people > Q: return False, float('inf') # overload
21            load += people
22            S[rid] = t # beginning of pick-up service (u_i)
23        else: # delivery
24            if t - (S[rid] + d) > t_ij(origin, destination) + THETA: # over maximum travel
25                time
26                return False, float('inf')
27            load -= people
28            z += (S[rid] - tau) + (t - tau - t_ij(origin, destination)) # pick-up delay+

```

```

    arrival delay
    c = nu; d_prime = d
    if T_prime and t + d_prime + t_ij(c, depot) > T0 + T: # getting back to the depot within
        the time window
        return False, float('inf')
    return True, z

def total_delay(routes, depots, Q, W, THETA, d, T0, T, people): # global objective: time
    delay of all customers
    return sum(evaluate(routes[k], depots[k], Q, W, THETA, d, T0, T, people)[1] for k in
        range(len(depots)))

# ----- INSERTION -----
def Insertion(requests, depots, *, Q=30, W=20, THETA=30, d=0, T0=6*60, T=4*60, PAX=1): # the
    asterisk ensures that the parameters after it are keyword only, so that they cannot be
    swapped by mistake
    routes = [[] for _ in depots] # initialization: K empty tours
    A = []; R = [] # lists of accepted/rejected requests
    for rid, o_name, dd_name, tau in requests: # process in the given order
        o = index[o_name]; dd = index[dd_name] # conversion dictionary from city names to
        indexes and vice-versa
        req = (rid, o, dd, tau)
        best_dz = float('inf'); best_k = None; best_route = None # Delta*, k*, T*
        for k, depot in enumerate(depots):
            Tk = routes[k]; L = len(Tk)
            _, z0 = evaluate(Tk, depot, Q, W, THETA, d, T0, T, PAX) # current tour cost
            for p in range(L + 1): # pick-up position insertion
                for q in range(p, L + 1): # delivery position insertion (>= pick-up one)
                    cand = Tk[:p] + [('P', o, req)] + Tk[p:q] + [('D', dd, req)] + Tk[q:]
                    ok, zc = evaluate(cand, depot, Q, W, THETA, d, T0, T, PAX) # evaluate the
                    route with the given insertion
                    if ok:
                        dz = zc - z0 # insertion cost
                        if dz < best_dz: # we consider the less costly insertion so far
                            best_dz = dz; best_k = k; best_route = cand
            if best_k is None:
                R.append(req) # no possible insertion: reject
            else:
                routes[best_k] = best_route; A.append(req) # accept less costly insertion overall
    return routes, A, R

# ----- OPTIMIZATION -----
def Optimization(requests, depots, N, *, Q=30, W=20, THETA=30, d=0, T0=6*60, T=4*60, PAX=1,
    seed=42): # the asterisk ensures that the parameters after it are keyword only, so that
    they cannot be swapped by mistake
    requests = list(requests) # work on a copy: the original list should not be reordered:
    the arrival order is meaningful
    rng = random.Random(seed)

```



```

61 routes_star, A_star, R_star = Insertion(requests, depots, Q=Q, W=W, THETA=THETA, d=d, TO
    =TO, T=T, PAX=PAX) # initialization: we consider the arrival order
62 z_star = total_delay(routes_star, depots, Q, W, THETA, d, TO, T, PAX)
63 for _ in range(N): # we consider (only) N reshufflings
64     for i in range(len(requests) - 1, 0, -1): # random permutation of the requests using
        the (optimized version of the) Fisher-Yates procedure. We are not interested in
        keeping the true arrival order, so we directly shuffle the original list
65         j = rng.randint(0, i) # uniform integer in [0, i] (endpoints included)
66         requests[i], requests[j] = requests[j], requests[i] # swap positions i and j
67     routes, A, R = Insertion(requests, depots, Q=Q, W=W, THETA=THETA, d=d, TO=TO, T=T,
        PAX=PAX)
68     z_val = total_delay(routes, depots, Q, W, THETA, d, TO, T, PAX)
69     if (len(A) > len(A_star)) or (len(A) == len(A_star) and z_val < z_star): # we seek
        to increase the number of served people, then to improve the cost (we look for
        little total delay)
70         routes_star, A_star, R_star, z_star = routes, A, R, z_val
71     return routes_star, A_star, R_star, z_star
72 # ----- string formatting -----
73 def read(request):
74     minutes = int(request[3])
75     time = f"{minutes//60:02d}:{minutes%60:02d}"
76     return f"{cities[request[1]]} -> {cities[request[2]]} @ {time}"

```

We now write down a datastructure which could be useful in a web based architecture that deals with the problem. This is a queue structure, whose job is to wait for new requests, feed them inside the insertion algorithm and, while the optimization algorithm is running, wait for other requests. The complication of which we were talking about in section 2.1 can be tackled simply by keeping track of the accepted requests and accepting an optimized version only if the previously accepted requests are not rejected in the new version. We can implement the queue structure as a class, specifically via a circular array. Here is the code:

```

1 class RequestsQueue:
2     """
3     Queue type class for the requests, implemented via circular array.
4     """
5     def __init__(self, capacity=500):
6         self._requests=[None]*capacity #pre-allocated array with fixed initial capacity
7         self._front=0 #front index
8         self._size=0 #number of current present requests
9         self._id=0 #progressive automatic id
10        self._capacity=capacity
11
12    def push(self, origin, destination, departure_time): #enqueue
13        if self._size == self._capacity: #if the capacity is reached, resize
14            print("Queue is full. Resizing...")
15            self.resize()
16            hours, minutes=departure_time.split(":")
17            self._requests[(self._front + self._size)%self._capacity]=[self._id, origin,
                destination, int(hours)*60+int(minutes)]
18            self._size += 1; self._id += 1

```

```

19     return
20
21 def pop(self): #dequeue
22     if self._size==0:
23         raise IndexError("Empty queue!")
24     r = self._requests[self._front]
25     self._requests[self._front]=None #empties the slot
26     self._front=(self._front+1)%self._capacity #update front
27     self._size-=1 #update size
28     return r
29
30 def resize(self):
31     old_capacity = self._capacity
32     temp = self._requests
33     self._capacity = 2 * self._capacity
34     self._requests = [None] * self._capacity
35     for i in range(self._size):
36         self._requests[i] = temp[(self._front + i) % old_capacity]
37     self._front = 0
38
39 def clear(self):
40     self._size=0
41     self._front=0
42     return
43
44 def is_empty(self):
45     return self._size==0
46
47 def is_full(self):
48     return self._size==self._capacity
49
50 def __len__(self):
51     return self._size
52
53 def __str__(self): #for the print method
54     if not(self.is_empty()):
55         print=""
56         for i in range(self._size):
57             j=(self._front + i) % self._capacity
58             time = f"{self._requests[j][3] // 60:02d}:{self._requests[j][3] % 60:02d}"
59             print+="id: "+str(self._requests[j][0])+", "+source: "+str(self._requests[j]
60                 [1])+", destination: "+str(self._requests[j][2])+" @ "+time+"\n"
61         return print
62     return ""

```

## 2.5 Solving the toy example with the proposed code

We can use the proposed code to solve the toy example using the following code:

```

1  # toy: (id, origin, destination, tau_in_minutes)
2  toy = [
3      (0, "Cosenza",      "Paola",      7*60),      # rho1, 7:00
4      (1, "Marano Marchesato", "Castrolibero", 7*60 + 5), # rho2, 7:05
5      (2, "Cosenza",      "Castrolibero", 7*60 + 5), # rho3, 7:05
6      (3, "Bisignano",     "Acri",      7*60 + 20), # rho4, 7:20
7  ]
8  toy_depots = [index["Rende"], index["Acri"]]
9
10 # 1) Insertion only (considering arrival order)
11 routes, A, R = Insertion(toy, toy_depots, Q=2, W=20, THETA=20, d=0, T0=7*60, T=70, PAX=1)
12 print("Insertion only")
13 print(" accepted:", [read(r) for r in A])
14 print(" rejected:", [read(r) for r in R])
15 print(" z =", total_delay(routes, toy_depots, 2, 20, 20, 0, 7*60, 70, 1))
16
17 # 2) Insertion + Optimization. The instance is tiny, so we enumerate ALL 4! = 24 arrival
    orders
18
19 best = None
20 for order in permutations(toy):
21     routes_o, A_o, R_o = Insertion(list(order), toy_depots, Q=2, W=20, THETA=20, d=0, T0
        =7*60, T=70, PAX=1)
22     z_o = total_delay(routes_o, toy_depots, 2, 20, 20, 0, 7*60, 70, 1)
23     if best is None or len(A_o) > len(best[1]) or (len(A_o) == len(best[1]) and z_o < best
        [3]):
24         best = (routes_o, A_o, R_o, z_o)
25 routes, A, R, z = best
26
27 print("Insertion + (exhaustive) Optimization")
28 print(" accepted:", [read(r) for r in A])
29 print(" rejected:", [read(r) for r in R])
30 print(" z =", z)

```

Note that we have first considered the insertion procedure only, which we know will yield only two accepted requests, and then the full insertion+optimization procedure.

In the latter, since there are few requests we manually enumerate all the arrival orders, without actually calling the `Optimization` function, which permutes randomly. The code yields the following results:

Insertion only

```

accepted: ['Cosenza -> Paola @ 07:00', 'Bisignano -> Acri @ 07:20']
rejected: ['Marano Marchesato -> Castrolibero @ 07:05',
          'Cosenza -> Castrolibero @ 07:05']
z = 25.114000000000007

```

Insertion + Optimization

```
accepted: ['Marano Marchesato -> Castrolibero @ 07:05',
          'Cosenza -> Castrolibero @ 07:05',
          'Bisignano -> Acri @ 07:20']
rejected: ['Cosenza -> Paola @ 07:00']
z = 59.265000000000164
```

## 2.6 Solving the full dataset with the proposed code

Analogously, we can use it to solve the problem posed by the full dataset. The full code is on the .ipynb file, here we suppose that we already have loaded the requests and the depots, of which we do a plot visible in figure 2.42:



**Figure 2.42:** Depots chosen as the highest-demand points in the clusters individuated via the use of the DBSCAN algorithm. The small, light blue dots are the other municipalities.

Here is the code to run solution on the full dataset:

```
# 1) Insertion only
```

```

2 fleet = [d for d in depots for _ in range(10)] # we simply (and stupidly) consider 10
   vehicles for each depot
3 routes, A, R = Insertion(requests, fleet, Q=10, W=20, THETA=20, d=0, T0=6*60, T=4*60, PAX=1)
4 z = total_delay(routes, fleet, 10, 20, 20, 0, 6*60, 4*60, 1)
5 print("Insertion only")
6 print(" accepted:", [read(r) for r in A])
7 print(" rejected:", [read(r) for r in R])
8 # ----- results -----
9 print(f" served {len(A)} / {len(requests)} (rejected {len(R)})")
10 print(f" total delay z = {z:.1f} min" + (f" ({z/len(A):.2f} min per served)" if A else ""))
11
12 # 2) Insertion + Optimization (reshuffling)
13 routes, A, R, z = Optimization(requests, fleet, N=10, Q=10, W=20, THETA=20, d=0, T0=6*60, T
   =4*60, PAX=1)
14 print("Insertion + Optimization")
15 print(" accepted:", [read(r) for r in A])
16 print(" rejected:", [read(r) for r in R])
17 print(" z =", z)
18
19 # ----- results -----
20 print(f"served {len(A)} / {len(requests)} requests (rejected {len(R)})")
21 print(f"total delay z = {z:.1f} min" + (f" ({z/len(A):.2f} min per served customer)" if A
   else ""))

```

which yields the following results:

Insertion only

```

accepted: ['San Marco Argentano -> Luzzi @ 06:12', 'Luzzi -> Castrolibero @ 07:35', 'Terrano
rejected: ['Corigliano-Rossano -> Rende @ 08:50', 'Corigliano-Rossano -> Praia a Mare @ 07:3
served 771 / 1000 (rejected 229)
total delay z = 3888.1 min (5.04 min per served)

```

Insertion + Optimization

```

accepted: ['Fiumefreddo Bruzio -> Corigliano-Rossano @ 06:36',
rejected: ['Rende -> Torano Castello @ 09:17',
z = 4375.5050000000001
served 779 / 1000 requests (rejected 221)
total delay z = 4375.5 min (5.62 min per served customer)

```

Where we can see that the optimization phase increases the number of answered requests slightly increasing the average delay time per person.

## Chapter 3: Analysis of the solution

### 3.1 Datastructures used

Throughout, we use the term array for Python's built-in `list`, which is internally a dynamic array: random access therefore take  $O(1)$  time.

The implementation of the algorithm proposed (for the offline case) relies on few simple data structures, namely:

- **the travel-time matrix** `distance_duration_table`, a  $150 \times 150$  table (array of arrays) whose entries  $(i, j)_{i,j=1}^{150}$  is the pair  $(\text{distance}_{ij}, \text{duration}_{ij})$ , where  $\text{distance}_{ij}$  is the distance (in kilometers) between city  $i$  and city  $j$  and  $\text{duration}_{ij}$  is the duration of the travel from city  $i$  to city  $j$ . Given the choice of objective function made, in the algorithm we are only concerned with the durations, this is why we use the accessor function `t_ij`, which returns just the duration. Since the feasibility assessment queries pairwise travel times, being able to quickly access these data (precisely, in  $O(1)$  time) is essential. The matrix is only built once during the creations of the datasets (see the appendix);
- **the name-index maps** `index`, a dictionary used to convert a city name to its integer position in the `cities_coordinates.txt` file and back, both in  $O(1)$ . This is done purely for convenience, since it is (reasonably) supposed that requests arrive as city names, but still, the algorithm works with integer indices;
- **the request array** `requests`, which in the offline mode holds all the requests, each request being a tuple  $(\text{id}, \text{origin}, \text{destination}, \tau)$ . A simple, plain array is the right structure for two reasons: the insertion heuristic iterates it in the given order, and the optimization level repeatedly considers random permutations of it: permuting efficiently requires random access in constant time;
- **the fleet** `depots` (or `fleet`), an array of depot indices, with one repetition of the corresponding depot index per vehicle;
- **the set of routes** `routes`, an array of  $K$  tours, one per vehicle. Each tour is an (ordered) array of stops, a stop being a triple of the type  $(\text{kind}, \nu, \text{req})$ , with  $\text{kind} \in \{ 'P', 'D' \}$  denoting a pick-up or a delivery,  $\nu$  being the city index, and `req` is the request the stop belongs to. Representing a tour as an array lets us build a candidate insertion by slicing, `Tk[:p] + [P] + Tk[p:q] + [D] + Tk[q:]`. Note that the flag lives on the stop, not on the city, meaning the same city may appear several times in a tour — as a pick-up for one request and a delivery for another;
- **the pick-up dictionary** `S`, local to `evaluate`, maps a request id to the time its pick-up service started. When the simulation later reaches that request's delivery, it retrieves the stored time in  $O(1)$  to check the maximum-ride-time constraint and to accumulate the customer's delay. A dictionary keyed by request id is needed because (if  $Q > 1$ ) several passengers can be on board at once, so each delivery must be matched to its own pick-up;
- **the accepted and rejected arrays** `A` and `R`, which collect the requests as they are decided. Appending is  $O(1)$  and they are read only at the end.

For the online mode, requests are not known in advance. They arrive over time, the order is important and can accumulate if the system is busy running the optimization algorithm. The natural structure then becomes a queue, implemented above as the class `RequestsQueue` via the usage of a circular array. In this way inserting a new request and extracting the next one are both  $O(1)$  time operations. In the implementation, a simple

resizing method in case the capacity of the queue is met is provided. In the running mode the queue plays the role the request list plays offline: it feeds requests one at a time to the insertion algorithm, and while the optimization (reshuffling) runs between two arrivals it stores the requests that keep coming in. The extra online constraint discussed in section 2.1 — that a re-optimized solution may not drop a request already confirmed to a customer — might be handled outside the queue, by keeping the set of accepted requests and discarding any reshuffled solution that fails to serve all of them.

## 3.2 Asymptotic complexity

We now analyze the computational complexity of the proposed solution. Throughout:

- $n$  is the number of requests;
- $K$  is the number of vehicles;
- $N$  is the number of reshufflings;
- $L_k^{(r)}$  is the number of stops in the tour  $k$  when request  $(r)$  is inserted. Since this can only grow as requests are inserted, it makes sense to uniformly bound all  $L_k^{(r)}$  with  $L := \max_{k,r} L_k^{(r)}$ , i.e. the maximum number of stops that is reached by any tour during the execution of the algorithm, so to consider the worst case scenario.

### 3.2.1 Single functions

- `t_ij` just accesses the elements of the distance-duration table, hence its time complexity is  $O(1)$ ;
- `evaluate` scans once the tour it receives. For each stop of the tour, the computational cost of the operations it does (specifically, some `t_ij` calls, a fixed number of comparisons and insertion operation on the dictionary `S` that takes constant time) is  $O(1)$ . On a tour with  $\ell$  stops, it takes  $O(\ell)$ ;
- `total_delay` calls `evaluate` once per vehicle. Its cost can be bound in two ways according to how one reasons:
  - since the routes contain at most  $n$  accepted requests, or equivalently at most  $2n$  stops in total. The costs is thus  $O(n)$  in the worst case;
  - since there are  $K$  total routes – one per vehicle – and each tour has at most  $L$  stops, the cost is  $O(KL)$  in the worst case;
- `Insertion`. For each of the  $K$  vehicles, when inserting request  $r$  it enumerates all positions  $(p, q)$  with  $0 \leq p \leq q \leq L_k^{(r)}$ . These are  $\binom{L_k^{(r)}+2}{2} = \frac{(L_k^{(r)}+2)(L_k^{(r)}+1)}{2}$  candidates, thus  $O((L_k^{(r)})^2)$  computational cost arises from here. For each of these candidates, the algorithm builds the tour by slicing the old one, an operation taking  $O(L_k^{(r)})$  time, and evaluates it, an operation that, as we have discussed before, when request  $r$  is being inserted has  $O(L_k^{(r)})$  cost. The cost for each vehicle is thus  $O((L_k^{(r)})^2) \cdot O(L_k^{(r)}) = O((L_k^{(r)})^3)$ , and over the whole fleet, bounding every  $L_k^{(r)}$  by  $L$ ,

$$O\left(\sum_{k=1}^K (L_k^{(r)})^3\right) = O(K L^3).$$

Since the full run of the `Insertion` function has to consider all  $n$  requests, the total final cost is  $O(n K L^3)$ .

### 3.2.2 Offline mode

**Optimization** basically runs the **Insertion** algorithm once on the arrival order and then once per reshuffling. Reshuffling the  $n$  requests is done via the Fisher-Yates procedure and takes  $O(n)$ . Note that since the reshufflings are uniform and we do not need to keep the arrival order, we do not need to copy the original arrival order and we can simply keep reshuffling the same list. Even if we needed to keep the original arrival order, the **Insertion** algorithm execution would still dominate. The **Optimization** algorithm costs:

$$O((N + 1) n K L^3) = O(N n K L^3),$$

with the  $O(Nn)$  spent shuffling being a lower-order term.

A remark which could be useful to get rid of the parameter that we can not control directly, namely  $L$ , note that this, i.e. the maximum number of stops that is reached by any tour, is bounded because of the shift start and end time constraints. This means that the **Optimization** algorithm cost has degree 1 – i.e. is linear – in  $n$ ,  $K$  and  $N$ .

As for space complexity, besides the fixed  $150 \times 150$  travel-time matrix, we store the requests array ( $O(n)$ ); the routes, which, following the same argument as above:

- contain at most  $2n$  stops in total, thus are  $O(n)$  in space;
- since there are  $K$  total routes – one per vehicle – and each tour has at most  $L$  stops, the cost is  $O(KL)$ .

one transient candidate tour ( $O(L)$ ); the lists of accepted/denied requests ( $O(n)$ ) and several constants. Assuming that  $n \gg K$  (and recalling that obviously  $L \leq n$ ), the space complexity is therefore  $O(n)$ .

### 3.2.3 Online mode

The queue structures supports **push** and **pop** of incoming requests having  $O(1)$  cost. When a request arrives, the **Insertion** algorithm, as it was described above, places it into the current routes at cost  $O(KL^3)$ . This is independent of  $n$  and has the aim of providing a fast answer to the customer. The optimization level, which – as it has been said many times – runs between the arrival of a request and the next one, reshuffles the whole set of the previously accepted requests only and runs again the insertion on it. We stress again that this is the main difference with the offline case: if a request is accepted, the algorithm never come back questioning this fact. Practically, this has to be assured because it would be quite strange to first tell someone that their request has been accepted and then deny it. If  $A_i \leq i$  requests have been accepted by the  $i$ -th arrival, following a reasoning similar to the one made above, this procedure costs  $O(A_i K L^3)$  per reshuffling. Exploring  $N$  reshufflings thus costs  $O(N A_i K L^3)$ . Assuming  $n$  total requests arriving to the system, the total work is:

$$\sum_{i=1}^n O(N A_i K L^3),$$

which bounding  $A_i \leq n$  yields:

$$O(N n^2 K L^3).$$

Note that in this case we have  $n^2$  and not  $n$ : in the online version, we keep optimizing a set – that of accepted requests – that keeps growing, instead of optimizing once all the requests. Obviously, in most applications the case in which  $A_i$  is exactly  $n$  is not so plausible, still we favour a lot of accepted requests, for obvious reasons. Furthermore, the parameter  $N$  has to be tuned based on the system's computational power, bearing in mind that the higher the  $N$ , i.e., the higher the reshufflings explored, the better the solution. The most important observation though is that in the online setting we only reshuffle the active requests, i.e. those that have not yet been completed. The worst (very not plausible) case remains the one discussed, but this is the crucial thing that



makes this type of solution good for a web-based application.

The space usage is still  $O(n)$ , given that the only things that change – the pending queue and the accepted request list – are  $O(n)$  in space.

## Chapter 4: Simulations

### 4.1 Online setting

We have presented above a datastructure which might be used to store and feed to the algorithms the requests that arrive to the system, i.e. the class `RequestsQueue`. Here we report the code used to run a simulation in the online setting and comment the results obtained.

First, we need the function to load the queue:

```
1 def load_queue(queue, path_file, limit=None):
2     """
3     Returns a RequestsQueue instance "queue" filled with the requests in the file path_file.
4     It is built to work with the requests in the file requests.txt, thus the format it
5     expects is:
6     one request per line of the type 'origin->destination->Hour:Minutes')
7     Optionally, load the first 'limit' many.
8     """
9     with open(path_file) as f:
10         next(f) # skip the header
11         i=0
12         for line in f:
13             if not line:
14                 continue
15             origin, destination, departure = line.split(">")
16             queue.push(origin, destination, departure)
17             i+=1
18             if limit is not None and limit==i:
19                 break
20     return
```

Secondly, we report the slightly modified version of the function `Insertion` needed. Indeed, in this offline setting this always starts from `K` empty tours, building a solution from scratch. This is possible because having the requests beforehand gives us the possibility to investigate the best order overall without having to accept or rejected requests without knowing how this will condition the overall solution. Online, each new request has to be placed into the plan already built for the previously accepted ones because we cannot discard these. The modification is in the new keyword `routes`. If this is not passed, the algorithm works exactly as before. In this way, the new request is inserted independently of how many requests have already been accepted: this is the fast answer needed.

```
1 # ----- INSERTION -----
2 def Insertion(requests, depots, *, routes=None, Q=30, W=20, THETA=30, d=0, T0=6*60, T=4*60,
3     PAX=1):
4     if routes is None: # <<< CHANGED: seed from a given plan, or start empty
5         routes = [[] for _ in depots] # initialization: K empty tours
6     A = []; R = [] # lists of accepted/rejected requests
7     for rid, o_name, dd_name, tau in requests: # process in the given order
8         o = index[o_name]; dd = index[dd_name] # conversion dictionary from city names to
```

```

indexes and vice-versa
8 req = (rid, o, dd, tau)
9 best_dz = float('inf'); best_k = None; best_route = None # Delta*, k*, T*
10 for k, depot in enumerate(depots):
11     Tk = routes[k]; L = len(Tk)
12     _, z0 = evaluate(Tk, depot, Q, W, THETA, d, T0, T, PAX) # current tour cost
13     for p in range(L + 1): # pick-up position insertion
14         for q in range(p, L + 1): # delivery position insertion (>= pick-up one)
15             cand = Tk[:p] + [('P', o, req)] + Tk[p:q] + [('D', dd, req)] + Tk[q:]
16             ok, zc = evaluate(cand, depot, Q, W, THETA, d, T0, T, PAX) # evaluate the
17                 route with the given insertion
18             if ok:
19                 dz = zc - z0 # insertion cost
20                 if dz < best_dz: # we consider the less costly insertion so far
21                     best_dz = dz; best_k = k; best_route = cand
22 if best_k is None:
23     R.append(req) # no possible insertion: reject
24 else:
25     routes[best_k] = best_route; A.append(req) # accept less costly insertion overall
return routes, A, R

```

Finally, we report the function needed to actually run the simulation:

```

1 def online_cm(queue, depots, *, N=10, Q=10, W=20, THETA=20, d=0, T0=6*60, T=4*60, PAX=1):
2     routes = [[] for _ in depots] # this will store the current plan, which we store across
3     arrivals
4     accepted, rejected = [], []
5     while not queue.is_empty():
6         req = queue.pop() # simulation of how a request arrives over time
7         # level 1: insert ONLY the new request into the current plan. Fast answer (O(K L^3))
8         routes, served, _ = Insertion([req], depots, routes=routes, Q=Q, W=W, THETA=THETA, d=
9             d, T0=T0, T=T, PAX=PAX)
10        if served: # If feasible, accept. This is immediately told to the customer and
11            irrevocable
12            accepted.append(req)
13            # level 2: between-arrival reshuffling of everything accepted. The best plan is
14            kept
15            r2, A2, _, z2 = Optimization(accepted[:], depots, N, Q=Q, W=W, THETA=THETA, d=d,
16                T0=T0, T=T, PAX=PAX)
17            if len(A2) == len(accepted) and z2 <= total_delay(routes, depots, Q, W, THETA, d,
18                T0, T, PAX):
19                routes = r2 # adopt only if it drops no previously accepted request and does
20                    not worsen z
21            else:
22                rejected.append(req) # infeasible -> rejected (irrevocably)
23    return routes, accepted, rejected

```

Note that we are not able to simulate the time passing and the requests being completed, and furthermore we empty the queue before being able to print something, thus we see the results only after several minutes,

according to the parameters chosen. The following results have been obtained after 2 minutes choosing 3 vehicles per depot and `N=5`:

Online simulation

```
served 160 / 200 requests (rejected 40)
total delay z = 363.8 min (2.27 min per served customer)
  accepted: ['San Marco Argentano -> Luzzi @ 06:12',
  rejected: ['Corigliano-Rossano -> Rende @ 08:50',
```

Despite the naive assignment of the vehicles to the depots, we still get an 80% acceptance rate. Furthermore, we get an average delay of 2.27min, which is considerably smaller than the one we obtained solving the full problem in the previous section, but to get a fair comparison we should run the `200` considered requests in the offline setting using the same parameters. Doing so, we get these results:

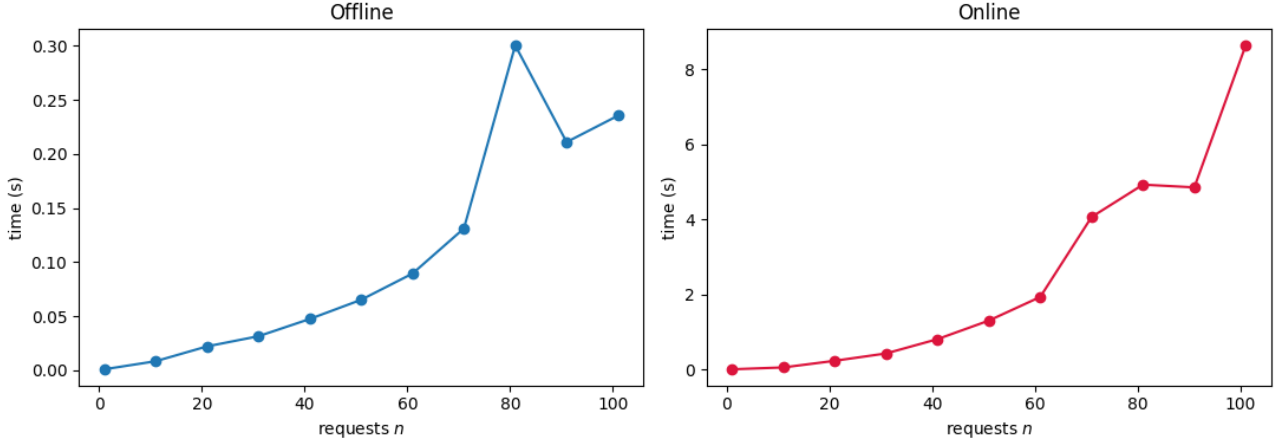
Insertion + Optimization

```
  accepted: ['Montalto Uffugo -> Trebisacce @ 06:14', ...
  rejected: ['Corigliano-Rossano -> Praia a Mare @ 07:32', ...
  z = 255.73700000000002
served 167 / 200 requests (rejected 33)
total delay z = 255.7 min (1.53 min per served customer)
```

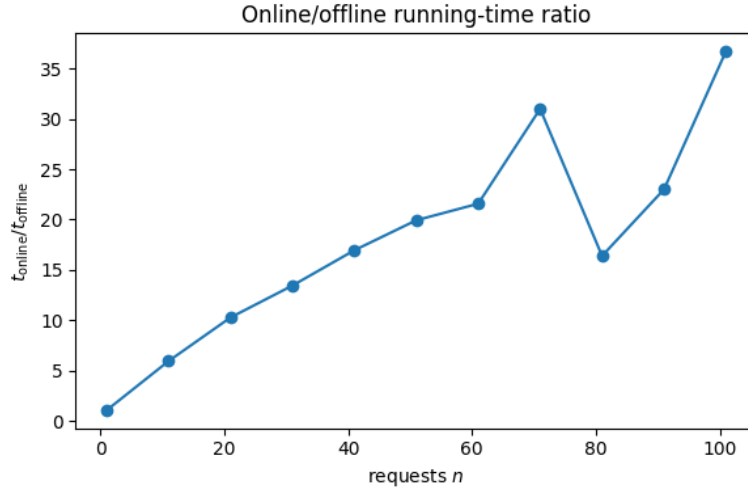
and these are in perfect accordance with the previously made analysis: in the offline setting, the algorithm is better qualitatively (7 requests more and a lower per-customer average delay) and far better regarding the time complexity (on our machine, this run only took a few seconds).

## 4.2 Running time versus number of requests

Beyond the quality of the produced solution, it is instructive to measure how the running time of the two settings grows with the number of requests. We let  $n$  range over an interval and, for each value, we time both the offline `Optimization` and the online `online_cm` on the first  $n$  requests, keeping the same fleet (3 vehicles per depot) and the same parameters. We fix `N=1`: the number of reshufflings enters the cost only as a constant multiplicative factor, so it does not change the *shape* of the curves, and keeping it minimal lets us reach larger values of  $n$  without waiting too long — the online routine being by far the more expensive of the two. At the same time we also record the acceptance rate and the average delay per customer of the two settings. For the code, check the `.ipynb` file.



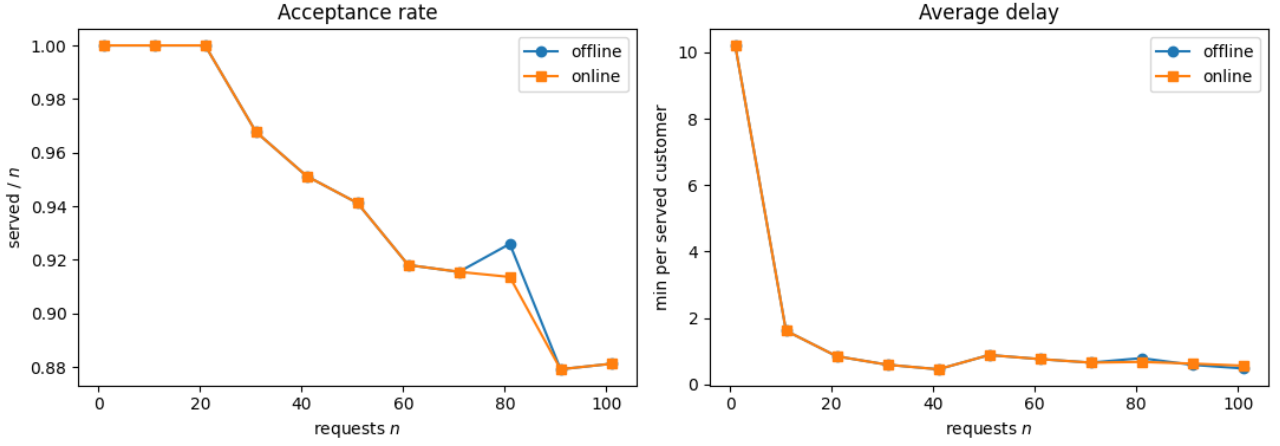
**Figure 4.1:** Running time (in seconds) of the offline and of the online setting as function of the number of requests  $n$ . Results on the same instances with 3 vehicles per depot and  $N=1$ .



**Figure 4.2:** Ratio of the online to the offline running time as a function of  $n$ . Roughly linear growth, giving an empirical result that is in accordance with the extra factor of  $n$  that separates the two costs found in the complexity analysis.

Note that both curves (Figure 4.1) grow more than linearly. This is expected: the bounds  $O(N n K L^3)$  and  $O(N n^2 K L^3)$  are of degree 1 and 2 in  $n$  at fixed  $L$ , but here the fleet is kept fixed while  $n$  grows, so the tours increase in length and  $L$  – the quantity that appears in the complexity analysis – increases with  $n$ , bending the offline curve upward as well. What really matters is that the online time lies far above the offline one and grows faster: their ratio increases roughly linearly with  $n$  (Figure 4.2), which is precisely the extra factor of  $n$  separating the two costs.

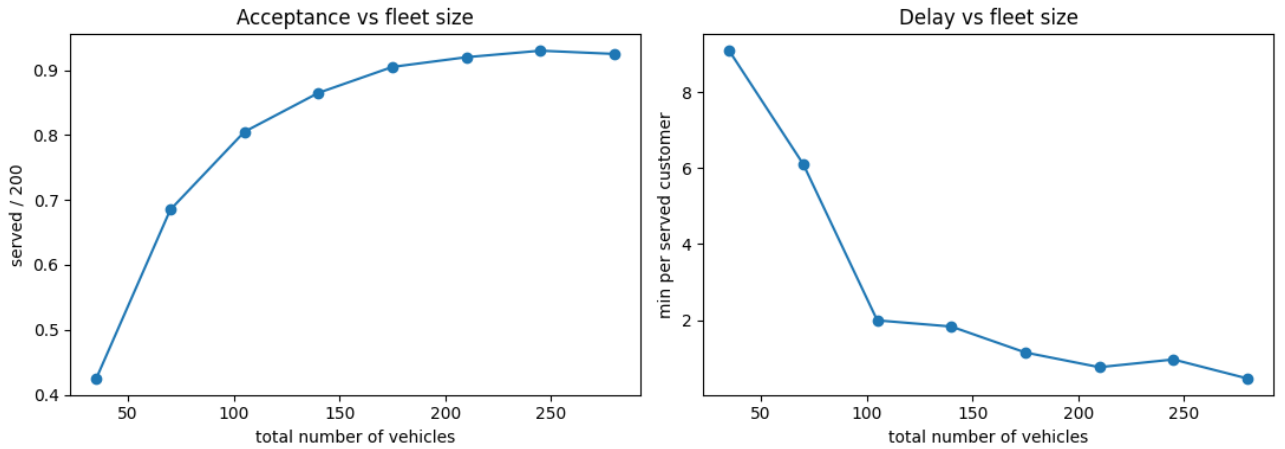
The acceptance rate and the average delay are reported in Figure 4.3. As we expect, the online mode never really outperforms the offline one: its acceptance rate stays at or below the offline one – the offline setting, knowing all the requests in advance, serves at least as many customers – while its average delay per customer tends to stay at or above the offline one, though more erratically, since this average is taken over the served customers only and the online mode serves fewer of them.



**Figure 4.3:** Acceptance rate (left) and average delay per served customer (right) in the offline and online settings, as a function of the number of requests  $n$  (same instances, 3 vehicles per depot,  $N=1$ ).

### 4.3 Fleet size versus acceptance

We finally look at how the fleet size affects the number of accepted requests and the average delay per customer. We consider only the offline setting and a naive assignment of the vehicles – the same number of vehicles at every depot, regardless of its demand – just to get a grasp of the relationship. Again, for the code, more on the .ipynb file.



**Figure 4.4:** Acceptance rate (left) and average delay per served customer (right) as a function of the total number of vehicles, on a fixed instance of 200 requests in the offline setting, with the same number of vehicles at every depot.

As we might expect, increasing the fleet size increases the number of requests the system can accept, even though the assignment is very naive, and the average delay decreases as well (Figure 4.4). Note, though, that even by increasing the fleet indefinitely it does not seem that we are able to serve all customers or lower the average delay arbitrarily. This is for two reasons:

- there are requests that are intrinsically hard, even impossible – for instance, ones whose origin cannot be reached within the pick-up window from any depot or whose destination cannot be reached respecting the end of the shift time, from every depot;
- the depots are fixed, and some requests may depart from a city whose closest depot is not really close, so the vehicle must first travel there. It is thus inevitable to accumulate some delay.

## Chapter 5: Conclusion

In this simplified application of the strategy individuated in [3] to tackle the DRT problem in its previously described declension, we have made a lot of simplifying assumptions, namely:

1. we have considered, because of a lack of "real" information, a "fake" dataset, with a generation of the requests process that takes into account very few factors than there are in reality;
2. we have assigned a constant number of vehicles to each depot;
3. since this was not the main objective of this project, we have chosen the depots in a fairly naive way using a simple DBSCAN procedure: we create neighbourhoods of sufficiently demand-packed, near cities and then select the busiest city of the neighbourhood as corresponding depot, labelling as noise very far or low-demand cities;
4. we have assumed that our fleet is homogeneous, meaning that each vehicle has the same capacity;
5. we do not consider traffic or other real-time information in the evaluation of the routes;
6. we have assumed that the customers are willing to bear at most 20min of pick-up delay and 20min of excess ride time compared to direct travel, for a worst-case scenario, total delay of 40min, which is arguably a lot;
7. we have assumed that the customer can only specify the desired pick-up time;
8. we only consider one stop per city, its centroid;
9. we have assumed symmetric travel times and distances;
10. in the process of generating the requests, we have consider a uniform probability of choosing the desired departure time;
11. we have assumed a constant (actually 0, but modifiable) service time per stop;
12. we have considered only 1 customer per request;
13. the objective considered is only (order matters!) total requests served and customers' delay.

Let us discuss the problems posed one by one.

The first element of the previous list does not invalidate the quality of the algorithm proposed, because this is somewhat quality-of-the-requests agnostic. In fact, the routes are built on the requests whatever the requests themselves. Yet, to study if this kind of solution would be really applicable to the province of Cosenza, it would be very insightful to get a grasp on what is the real distribution of the requests among the province, thus it might be a good idea to make polls, and where there is a poll to be made, a researcher has to be scared.

The second element of the previous list is actually something to be tuned, and one might think, trivially, to assign more vehicles to depots with higher demand, or more vehicles to "more central", well-connected depots.

the third element of the previous list is a hard problem: it makes sense to use a clustering, density-based algorithm such as DBSCAN because in this case we have a clear interpretation of what "density" means here: we want "tiny" zones with many requests to be clustered together and identified by a certain point, for example (the centroid of) the busiest city of the cluster. Yet, this is probably not the best solution and in order to be more accurate one should empirically study how the objective function changes when all other parameters are kept constant and the depots are vary.

The fourth element of the previous list does not pose that much of a challenge, given that the algorithm can easily accomodate diversity among the vehicles, whose capacities would just need to be stored and used when necessary. Nevertheless, it might be an important point to investigate because it might happen that if requests from some low demand depot tend to have really distant cities as destination, a bigger vehicle capable of serving

---

lots of stops might be preferred, and similarly if requests from some high demand depot tend to have near cities as destination, many smaller vehicles able to parallelize the serving of the requests might be preferred.

The fifth element of the previous list is a very difficult problem, but the information gathered would be easy to exploit in algorithm proposed.

The sixth element of the previous list sits at the very bottom of the DRT implementation problem and is one of the main features in the creation of the algorithm. Deciding what to leave to the choice of the customers and what to "hardcode" in the algorithm is not an easy task. Again, we have followed [3], but other approaches are possible.

The seventh element of the previous list is strongly connected to the previous one and has to do with personalizability choices in the design of the model.

The eighth element of the previous list could be easily solvable if one had the time and the resources to decide what places to include and look for them.

The ninth element of the previous list is somewhat connected to the fifth one. Indeed, other than just thinking about the fact that if for going from A to B one follows a certain path it is not the case that to get back to A from B they have to follow the same path, one could go a step further and think about the fact that travel times strongly depend on the conditions of the road to be made, and these are intrinsically real-time information that cannot be (that accurately, at least) predicted.

The tenth element of the previous list is somewhat connected to the first one. Indeed, we are again talking about questionable modelling choice which could be refined or, even better, hardcoded after an empirical study.

The eleventh element of the previous list is not that much of a problem and one could think of being overconservative and set a constant service time for each stations that is presumably greater than the real one for each station or, again, do an empirical study and then assign a specific service time duration to each stop.

The twelfth element would be fairly simple to include as feature in the algorithm: if a request comes with an attached number of customers, then we just need to record this information and change the definition of request from 3-tuple to 4-tuple.

The thirteenth element is not really a problem: as long as we have a functional expression for the objective, we can choose the "best solution" based on the objective itself. The problem is shifted to designing a good objective. For example, considering only total number of customers served and total customers' delay makes the job, but in reality is pretty unrealistic: we are basically favoring very costly (in terms of distance run by the vehicles) solutions in order to increase the number of customers served, and after that we only care about the total delay of the passengers of the vehicle, not at all about the cost of the travel.

These comments can all be thought of possible ways of improving the algorithm, whose solution, which we recall being heuristic, might be greatly improved even by some relatively simple change.

Let us now comment on the 1000 performance of the algorithm on the requests generated by our model. First of all, note that  $771/1000 = 77.1\%$  of the requests have been served with the `Insertion` algorithm only, while the number increases to  $779/1000 = 77.9\%$  if we consider the `Optimization` routine. We can note that the time of execution, even considering just  $N = 10$  reshufflings of the original requests, is high (on our machine, 2min28.4sec). This is because in the offline mode, in which the requests do not get served and thus their number stays high, the cost of reshuffling and inserting to optimize the found solution is very high, and also assigning without any caution 10 vehicles per depot produces  $10 \times 35 = 350$  vehicles/routes to try each time for the insertion and to evaluate, which clearly adds to the time cost. Some important improvements in the complexity of the algorithm could be made using the concepts of "forward time slacks" and related eighth step procedure reported, for example, in [4] and also the "neighbourhood reduction" procedure. These consist,



---

respectively, in:

- creating the schedule from a precomputed time slack – which can be described as the largest amount by which the start of service at a stop can be delayed without making any time window or ride-time violation worse – rather than re-simulating the whole tour from scratch at every tentative insertion. This is because when the route’s stops are fixed, the times are not and different times lead to different routes, with one that could be the actual best one;
- fixing the time window critical stop of a request at its best position before placing its companion stop. In other words, this means that (in our case) we first fix the best position in the route for the pick-up stop (since it is the one the customers have a say on) having to look through  $O(L)$  stops, and then fix the delivery stop looking through the ones following, another  $O(L)$  operation.

These allow, respectively, evaluating (and optimally timing) a candidate route linearly in its length instead of re-scanning it for every candidate, and reducing the number of insertion positions examined per request from  $O(L^2)$  to  $O(L)$ . Together they remove a factor of  $L$  from the insertion cost of each request.

## Bibliography

- [1] Hugo Barbosa et al. “Human mobility: Models and applications”. In: *Physics Reports* 734 (Mar. 2018), pp. 1–74. ISSN: 0370-1573. DOI: [10.1016/j.physrep.2018.01.001](https://doi.org/10.1016/j.physrep.2018.01.001). URL: <http://dx.doi.org/10.1016/j.physrep.2018.01.001>.
- [2] László Buics, Adhie Prayogo, and Boglárka Eisingerné Balassa. “The Role of Demand-Responsive Transport Systems in Sustainable Urban Mobility: A Systematic Literature Review and Stakeholder Analysis”. In: *Engineering Proceedings* 79.1 (2024). ISSN: 2673-4591. DOI: [10.3390/engproc2024079040](https://doi.org/10.3390/engproc2024079040). URL: <https://www.mdpi.com/2673-4591/79/1/40>.
- [3] Pasquale Carotenuto and Fabio Martis. “A double dynamic fast algorithm to solve multi-vehicle Dial a Ride Problem”. In: *Transportation Research Procedia* 27 (2017). 20th EURO Working Group on Transportation Meeting, EWGT 2017, 4-6 September 2017, Budapest, Hungary, pp. 632–639. ISSN: 2352-1465. DOI: <https://doi.org/10.1016/j.trpro.2017.12.131>. URL: <https://www.sciencedirect.com/science/article/pii/S2352146517310281>.
- [4] Jean-François Cordeau and Gilbert Laporte. “A tabu search heuristic for the static multi-vehicle dial-a-ride problem”. In: *Transportation Research Part B: Methodological* 37.6 (2003), pp. 579–594. ISSN: 0191-2615. DOI: [https://doi.org/10.1016/S0191-2615\(02\)00045-0](https://doi.org/10.1016/S0191-2615(02)00045-0). URL: <https://www.sciencedirect.com/science/article/pii/S0191261502000450>.
- [5] Jean-François Cordeau and Gilbert Laporte. “The dial-a-ride problem: models and algorithms”. In: *Annals of Operations Research* 153 (2006), pp. 29–46. URL: <https://api.semanticscholar.org/CorpusID:31161830>.
- [6] Peiqing Li et al. “Demand Response Transit Scheduling Research Based on Urban and Rural Transportation Station Optimization”. In: *Sustainability* 14.20 (2022). ISSN: 2071-1050. DOI: [10.3390/su142013328](https://doi.org/10.3390/su142013328). URL: <https://www.mdpi.com/2071-1050/14/20/13328>.
- [7] Stephanie E. Schasché, Robert G. Sposato, and Nina Hampl. “The dilemma of demand-responsive transport services in rural areas: Conflicting expectations and weak user acceptance”. In: *Transport Policy* 126 (2022), pp. 43–54. ISSN: 0967-070X. DOI: <https://doi.org/10.1016/j.tranpol.2022.06.015>. URL: <https://www.sciencedirect.com/science/article/pii/S0967070X22001809>.

## Appendix: Constructing the datasets

In this appendix, we will see how the files `cities_coordinates.txt`, `distance-duration_table.txt` and `requests.txt` have been created. These store respectively:

- the coordinates of all the 150 cities in the province of Cosenza in the format:

City: (latitude, longitude);

- the distances - in kilometers - and the durations - in minutes - between the  $i$ -th city and the  $j$ -th, with  $i, j$  ranging from 1 to 150, in the following format:

City<sub>1</sub> : (distance<sub>1 2</sub>, duration<sub>1 2</sub>); (distance<sub>1 3</sub>, duration<sub>1 3</sub>); ... , (distance<sub>1 150</sub>, duration<sub>1 150</sub>)

City<sub>2</sub> : (distance<sub>2 3</sub>, duration<sub>2 3</sub>); (distance<sub>2 4</sub>, duration<sub>2 4</sub>); ... , (distance<sub>2 150</sub>, duration<sub>2 150</sub>)

⋮

City<sub>149</sub> : (distance<sub>149 150</sub>, duration<sub>149 150</sub>)

City<sub>150</sub> :

where we only store the distances and durations of travels from City <sub>$i$</sub>  with  $i \in \{1, \dots, 150\}$  to City <sub>$j$</sub>  with  $j \in \{i, \dots, 150\}$  because we are assuming that if City <sub>$i$</sub>  is  $d$  kilometers distant from City <sub>$j$</sub> , then City <sub>$j$</sub>  is  $d$  kilometers distant from City <sub>$i$</sub>  and a similar reasoning applies for the duration;

- 1000 hypothesized requests (since the total population of the province of Cosenza is 666516, this would correspond to  $\approx 0.15\%$  of the people answering an eventual poll which is reasonable) in the format:

Source->Destination->Desired departure time .

These have been generated according to a gravity model, see [1], section 4.2.1. Describing it briefly, this is a model developed to approximate human migration patterns between communities. Say that  $T_{ij}$  is the migration flow between community  $i$  and community  $j$ . Then this model supposes:

$$T_{ij} \propto \frac{P_i \cdot P_j}{r_{ij}},$$

where  $P_i$  is the population of community  $i$ ,  $P_j$  the one of community  $j$  and  $r_{ij}$  is the distance between community  $i$  and  $j$ . Generalizing:

$$T_{ij} = m_i \cdot m_j \cdot f(r_{ij}),$$

for a certain function  $f$  and masses (depending on the respective populations)  $m_i, m_j$ . The masses are related to the number of trips leaving  $i$  being attracted by  $j$ . In modelling the masses, one usually takes into account some socio-economic factors. We will make the masses depend on the population, the presence of universities, the presence of high-schools and the presence of railway stations in this way:

$$m_i = P_i + 25000U_i + 200H_i + 500R_i;$$

where, referring to city  $i$ :

- $P_i$  is its population;
- $U_i$  an indicator which is 1 if it contains a university and 0 otherwise;
- $H_i$  an integer-valued variables which says how many high schools (excluding "scuole serali" and counting all schools of the same institute separately) are there in it;
- $R_i$  is a categorical integer-valued variable taking values in  $\{0, 1, 2, 3, 4\}$ , with the meanings "no railway station", "local railway station", "local plus railway station", "plus railway station", "major

railway station" respectively;

so that one could interpret the formula as: the mass of a city is its population plus the number of students and workers a university brings plus the number of students and workers a high school brings plus the number of passengers that a railway station (of a certain level. Again, see the notebook for more precise informations) brings. To see how the specific weights of all this variables have been calculated, see the Colab notebook in the "Appendix: Constructing the datasets" section.

## 1.1 cities\_coordinates.txt

The first thing to do is to retrieve all cities in the province of Cosenza. We were able to do so by exploiting the website [istat.it](http://istat.it) of the national statistical institute of Italy, which stores various data regarding the country. In particular, the following [link](#) can be used to download a .csv file containing all municipalities of Italy, with information to the province to which they belong to. This is a very quick operation since the function just reads the provided file.

Now we need to **geocode** the cities, i.e. actually write the file `cities_coordinates.txt` described above inserting the name of each city and the respective coordinates in the format (latitude, longitude). This can be achieved using `geopy.geocoders`' `Nominatim`. More information is reported on the Colab notebook.

Once the access to the personal Google Drive storage is granted, the code used creates the `cities_coordinates.txt` file inside the Drive folder specified in the variable `path`. `Nominatim` is the geocoder for OpenStreetMap data. All documentation is in the sites indicated as comments in the code. In particular, an interesting question to ask and answer to is: what does the point on Earth's surface individuated by the returned coordinates correspond to for each city? The answer is: to the **centroid** of the administrative region of the city itself, as it is written [here](#) (search for **centroid** in the webpage). This is defined as the point whose coordinates are the averages of the coordinates of the points which fall inside the polygon representing said administrative region.

This chunk of code takes a long time running ( $\approx 26$  minutes). Indeed, a pause of 10 seconds between an API request (see the `geocode_city` function) and another one is needed (otherwise the server gives bad gateway!) and since there are 150 cities in total,  $150 \cdot 10s \approx 26min$ .

## 1.2 distance-duration\_table.txt

Now we discuss the code to create the file `distance-duration_table.txt` described above. This is based on the `table` service provided by the OSMR project developers (praise be to them!), and takes approximately 1min to run, which really is not that much. Indeed, the `table` service allows to evaluate the (by car) distance and duration of travel between multiple cities (both as sources and destinations) in a single API call. The real bottleneck is in fact the `time.sleep(10)` command in the nested loop (see the code in the Colab notebook) - which is needed in order to avoid server overload -, but being able to evaluate the distance and the duration of travel between multiple cities (both as sources and destinations) in a single API call allows to do less requests (in particular, given the chunk size - see the comments in the code and in particular the last line in the description of the function -, there are 6 API calls, thus  $6 \cdot 10s = 1min$ ). Fun fact, in an earlier version of the code I did not read through the documentation and ended up waiting something like an hour to get the distance table done, since I evaluated the (by car) distance using only one city as source.

## 1.3 requests.txt

We now turn to the generation of the requests file as described above. To do so, we will first create the file `cities_data.txt` containing the information of which we already discussed above (about population, universities, high schools and railway stations) in the following format:

- a header row:

#City;population;railway;university;high schools

- 150 successive rows (one for each municipality in the province of Cosenza, in the same order in which the cities are contained in the file `cities_coordinates.txt`) containing the data indicated in the header row. For example, the first of these 150 rows will be:

Acquaformosa;930;0;0;0.

The first thing we write about in the file is the population of all the cities. Data about this can be downloaded [here](#) in the section "Area download" and searching for "Cosenza" as a .csv file. Analyzing the file, one can see that the age value 999 is used as a flag for the rows storing the "total" values of the relative data. In the code, when we write about population we also initialize the columns `railway`, `university`, and `high schools` to the value 0 for all cities. This is because the data about these categories have been hand-written. In particular:

- the `railway` column has been retrieved from [this site](#) after having selected the "Lista" option and the region of interest (Calabria) as well as the province of interest, following the classification of RFI on the importance of the stations (and signaling the absence of railway stations) with the conversion: NO RAILWAY STATION->0, LOCAL->1, LOCAL PLUS->2, PLUS->3, MAJOR->4 (attention to the fact that RFI signals a LOCAL station in Corigliano and a LOCAL PLUS in Rossano. Since the two cities merged and apparently the RFI site has not been updated, we will flag Corigliano-Rossano as a LOCAL-PLUS hub);
- there is only 1 university in the province of Cosenza, as it can be retrieved from [this site](#) (official, by the Italian Ministry of University and Research MUR) and [this site](#) (unofficial, a Wikipedia page);
- the number of high schools per city has been retrieved from [this site](#) (official, by the Italian Ministry of Education and Merit MIM) after having selected "Ricerca avanzata" (Advanced research), "Cerca per: Regione, provincia o comune" (Search for: Region, province or municipality), "Regione: CALABRIA" (Region: Calabria), "Provincia: Cosenza" (Province: Cosenza), "Comune: Tutte le opzioni" (Municipality: All the options), "Tipo di istruzione: Scuola secondaria di II grado" (Education level: high school), "Percorso: Tutte le opzioni" (Path: All the options), "Tipologia scuola: Statale Paritaria" (Type of school: Statal Equalized, tick both options). We can see that this research yields 172 found high schools, but from this we decided to exclude the "Scuole serali". Furthermore, for each "Istituto", institute, we count every site/school. Artificial Intelligence (specifically, Claude Opus 4.8 max) has been used to automatize this process: 132 high schools have been individuated by said AI model.

To come back to the matter of the "same order as in the file `cities_coordinates.txt`", (and thus also in `distance-duration_table.txt`), we manually move the rows corresponding to `Casali del Manco` and `Corigliano-Rossano` to the bottom of the file. This is because in the generation of the file `cities_coordinates.txt` an [istat.it](#) dataset in which these cities where respectively second-to-last and last element was used.